MediLens — End-to-End Healthcare Project Plan

1. Overview and vision

MediLens is a web app that helps patients **understand their own medical records**: scan the record (OCR) → AI extracts diagnoses, lesion locations and measurements → shows them on a 3D/2D body model and organ diagrams per department, with a before/after timeline. The project is run like an enterprise project — full process, documentation, testing, security and operations — but one person plays every role.

Working name: MediLens (rename freely). **Suggested duration:** 9–12 months part-time (~20 h/week) or 5–6 months full-time.

Goals (measurable)

#	Goal	Success measure
G1	Patients upload a record and see the lesion on the body model	≥ 85% of test documents extracted and mapped to the correct body region
G2	Plain-language explanations with sources, never replacing a doctor	100% of AI answers carry citations + disclaimer
G3	Track progression over time (pre/post)	Timeline shows ≥ 2 points for the same finding
G4	Enterprise-grade system	CI/CD, test coverage ≥ 70%, 99.5% SLO, complete audit log
G5	You learn how large projects run	Full document set from section 12 (BRD, SRS, ADRs, runbooks…)

Scope

- **In scope:** auth and personal profiles, record upload and OCR, medical entity extraction, anatomical mapping, per-department 3D/2D viewers, timeline, LLM explanations with RAG, sharing with doctors/family, admin and audit.

- **Out of scope (first release):** diagnosis, prescribing, direct hospital HIS/EMR integration, advanced DICOM (CT/MRI) processing, insurance payments.

Project principles

1. **Safety first:** the app *illustrates and explains*; it does not diagnose. All AI content is labelled "AI-generated" with confidence and sources.
2. **Synthetic data first, real data later:** use synthetic/public data throughout development; real data only once security and consent are in place.
3. **Human-in-the-loop:** users always review and correct extraction results before saving.
4. **Everything documented and measurable:** every decision has an ADR, every feature has acceptance criteria and a metric.
5. **One hat at a time:** dedicate time each week to each role and produce exactly the output someone in that role would deliver.

2. Phase 0 – Ideation and discovery

Discovery takes 3–4 weeks and ends with an **approved Product Brief + BRD** (you approve as Product Owner using the checklist at the end of this section). No production code in this phase — prototypes only.

2.1 Problem statement

Patients receive records full of jargon, abbreviations and handwriting across many pages. They cannot picture *where the lesion is, how severe it is, how it affects other organs, or how it changed after treatment*. Doctors have little time to explain. MediLens turns the record into visuals plus sourced explanations.

2.2 Personas

Persona	Description	Main need	Pain
Patient	25–65, smartphone user, no medical background	Understand the illness, track progress	Hard jargon, anxiety, lost paper records
Caregiver	Child/parent caring for a patient	View a relative's record when permitted	No access, scattered information

Persona	Description	Main need	Pain
Clinician	Doctor the patient shares with	Quick summary, fix wrong extractions	Little time, unstructured data
Admin	Runs the system	Manage users, anatomy content, audit	Incidents, compliance requests
Medical Reviewer	Approves content (you play this role)	Approve condition templates, region mappings	Wrong content causes harm

2.3 Main user journey (Patient)

1. Sign up → verify email/OTP → accept terms and health-data processing (consent).
2. Create a health profile (sex at birth, birth year, height/weight — optional).
3. Upload record photo/PDF → image quality check → OCR → extraction.
4. **Review** screen: original on the left, extracted fields on the right; the user confirms or corrects.
5. The system assigns Department/Unit and marks the lesion on the 3D/2D model.
6. The user rotates/zooms the model, clicks a region → explanation: what it is, common causes, consequences, possible spread/metastasis (if any), questions to ask the doctor.
7. Uploading a later record → timeline comparing before/after.
8. Time-limited sharing with a doctor/relative; follow-up reminders.

2.4 MVP and later releases (MoSCoW)

Feature	MVP (v1.0)	v1.5	v2.0
Auth (email, Google OAuth), MFA	Must		
Photo/PDF upload, Vietnamese + English OCR	Must		
Extract diagnosis, location, severity, date	Must		
Review/correct screen	Must		
Full-body 2D + 3D viewer	Must		

Feature	MVP (v1.0)	v1.5	v2.0
First 3 units: Dental, Internal (Cardiology), Oncology	Must		
LLM explanations + RAG with citations	Must		
Pre/post timeline	Should		
Sharing with doctor/relative	Should		
More units: Surgery, Nephrology–Urology, Gastroenterology, Respiratory		Could	
Computer vision on lesion photos (skin, teeth)		Could	
Reminders and notifications		Could	
FHIR export/import, hospital integration			Won't (v1)
Native mobile app			Won't (v1)

2.5 Research to do

- **Competitors / references:** BioDigital Human, Visible Body, Complete Anatomy (3D anatomy); Ada, K Health (symptom checkers); personal health record apps in Vietnam. Record features, visual style, business model, weaknesses.
- **Interview 5–8 people** (friends, relatives who have been hospitalised): what is hard to understand in records? What would they want to see? What worries them about putting health data online?
- **Collect sample records** (redacted) to learn layouts: consultation forms, discharge papers, lab results, pathology reports, dental X-ray reports.
- **Legal:** Vietnam's Law on Personal Data Protection 91/2025/QH15 (in force 1 Jan 2026) and its guiding Decree 356/2025/ND-CP, which replaced Decree 13/2023 (health data is sensitive data); use HIPAA/GDPR as international references.

2.6 Phase 0 outputs

- [ ] Product Brief (1–2 pages): problem, users, value, North Star metric
- [ ] Personas + User Journey Map
- [ ] Competitive analysis
- [ ] BRD (Business Requirements Document) v1

- [ ] Lean Canvas / Business Model Canvas
- [ ] Click-through prototype (Figma) for upload → review → viewer
- [ ] Assumption list and how to test each
- [ ] Go/No-Go decision for the MVP

3. Business units (departments and sub-units) and domain model

The system has 3 levels: **Department → Unit (specialty/sub-unit) → Anatomical Region**. Each Unit has its own viewer, condition dictionary and mapping rules. The MVP builds 3 Units; the architecture must allow adding a Unit through configuration + assets only, without touching core code.

3.1 Departments and units

Department	Sub-units	Dedicated viewer	Key extracted data	Phase
Dental & Maxillofacial	General dentistry, Orthodontics, Maxillofacial surgery, Periodontics	2D 32-tooth chart (FDI 11–48) + 3D jaw model	Tooth number, surface (M/D/O/B/L), caries, missing, fillings, implants, periodontitis	MVP
Internal Medicine	Cardiology, Respiratory, Gastroenterology, Endocrinology, Neurology	3D heart (4 chambers, valves, coronaries), two lungs, GI tract	ICD-10 diagnosis, EF%, blood pressure, lab values	MVP (Cardiology), v1.5 for the rest
Surgery	Orthopaedics/Trauma, General surgery, Neurosurgery	3D skeleton, joints, incision sites on skin	Fracture/lesion site, left/right, procedure	v1.5
Nephrology–Urology (Excretory)	Nephrology, Urology	Both kidneys, ureters, bladder	eGFR, creatinine, stones (site, size)	v1.5
Oncology	Cancer by organ (lung, liver, breast, colorectal…)	Primary organ + lymph nodes + metastatic sites on full body	Tumour site, size, TNM, stage, pathology, metastases	MVP
Dermatology	General dermatology	Full-body 2D skin map (front/back)	Site, size, lesion type	v2.0 (with CV)

Department	Sub-units	Dedicated viewer	Key extracted data	Phase
Ophthalmology, ENT	Eye, ENT	3D eye, ear/nose/throat diagrams	Visual acuity, eye pressure, lesion site	v2.0

3.2 Unit configuration

Each Unit is described by a config file (JSON/YAML, stored in the DB with versioning):

- `unit_code`, `department_code`, VI/EN names, icon
- `regions[]`: anatomical regions, each with `region_code`, a standard reference code (e.g. FMA or SNOMED CT body structure), mesh ID in the 3D file, path ID in the 2D SVG, laterality (left/right/midline)
- `conditions[]`: conditions in the Unit, ICD-10 code, severity scale, default region, regions it can spread to
- `extraction_schema`: JSON Schema of fields the LLM must extract for this Unit
- `visual_rules`: colour by severity, marker type (filled region, point, line), animation
- `content_refs`: explanation articles in the knowledge base

3.3 Domain model (business entities)

Entity	Meaning	Key relationships
User	Login account	1 User – n Roles; 1 User – 1..n Patient Profiles (self + managed relatives)
PatientProfile	Health profile	n MedicalDocuments, n Conditions, n Observations
MedicalDocument	One upload (image/multi-page PDF)	n DocumentPages, 1 latest ExtractionRun
ExtractionRun	One AI pipeline run	Model version, prompt, raw output, confidence
Encounter	A visit/admission (date, facility)	Groups Conditions/Observations from one visit
Condition	Disease/diagnosis	Belongs to a Unit, has 1..n BodyFindings, status over time

Entity	Meaning	Key relationships
BodyFinding	Lesion on a body region	Region, severity, size, laterality, date observed
Observation	Measured value (lab, vital sign)	LOINC code (if mappable), value, unit, reference range
Department / Unit / Region	Anatomy and specialty catalogue	Hierarchy
KnowledgeArticle	Sourced medical explanation	Linked to Condition/Region, reviewer and review date
ShareGrant	Sharing permission	Grantee, scope, expiry, status
Consent	Data processing consent	Type, terms version, timestamp, withdrawal
AuditEvent	Access log	Who, what, which data, when, from where

Learning tip: name entities close to **HL7 FHIR** resources (Patient, Encounter, Condition, Observation, DocumentReference, Consent, AuditEvent). FHIR export becomes easy later and you learn the international health data standard.

4. Business rules

Each rule has a `BR-xxx` code traced to user stories, test cases and code (traceability matrix). Rules are written as "When… then… unless…" and each has an automated test.

4.1 Accounts and access

Code	Rule
BR-001	Users must be ⩾ 18 to self-register; child profiles can only be created by a guardian (Caregiver) and are flagged `minor`.
BR-002	Terms + health-data processing policy must be accepted before the first upload. A new terms version requires re-consent.

Code	Rule
BR-003	MFA is mandatory for Clinician and Admin; recommended for Patient.
BR-004	Lock the account for 15 minutes after 5 consecutive failed logins; send an email alert.
BR-005	Sessions expire after 30 minutes of inactivity; refresh tokens last at most 7 days and rotate on each use.
BR-006	RBAC + ownership checks (ABAC): a Patient sees only their own or shared profiles; Admins do **not** see medical content without a logged reason (break-glass).

4.2 Documents and AI extraction

Code	Rule
BR-010	Accepted formats: JPG, PNG, HEIC, PDF; ⩽ 20 MB/file, ⩽ 30 pages/document.
BR-011	Reject images whose blur score (Laplacian variance) is below threshold or whose short side is < 1000 px; show re-capture guidance.
BR-012	Every extracted field has a 0–1 confidence. Fields < 0.7 are highlighted yellow and **must** be confirmed by the user.
BR-013	AI results stay `draft` until the user confirms → `confirmed`. Only `confirmed` data appears on the viewer and timeline.
BR-014	All user edits are versioned (who, old/new value) and used for model evaluation (with consent).
BR-015	If a finding cannot be mapped to a body region, set `region = unmapped` and list it instead of guessing a location.

Code	Rule
BR-016	Duplicates: same file hash, or same visit date + facility + diagnosis → duplicate warning, no new record.
BR-017	Unit classification uses an ICD-10 → Unit mapping table; a condition belonging to several Units (e.g. lung cancer → Oncology + Respiratory) appears in both, with Oncology as primary.

4.3 Display and medical content

Code	Rule
BR-020	One severity palette system-wide: green (monitor), yellow (mild), orange (moderate), red (severe), purple (metastasis/spread). Always paired with a symbol/text label, never colour alone (accessibility).
BR-021	Metastasis/spread is shown only when the record states it (e.g. M1, "liver metastasis"). "Possible spread" is general educational content, shown separately with the label "reference information, not your condition".
BR-022	The LLM answers only from the approved knowledge base (RAG) + the user's own data; with no source it replies "no information yet, please ask your doctor".
BR-023	The LLM never gives doses, never advises stopping/changing medication, never predicts survival time. Emergency questions (chest pain, breathlessness, self-harm intent) → show emergency number 115 and advise seeking care immediately.

Code	Rule
BR-024	Every KnowledgeArticle needs sources (WHO, Ministry of Health, NIH/MedlinePlus, specialty guidelines), a reviewer and a review date; after 12 months it is flagged for re-review.
BR-025	Pre/post comparison only between BodyFindings of the same Region and Condition; different units of measure → convert or don't compare.

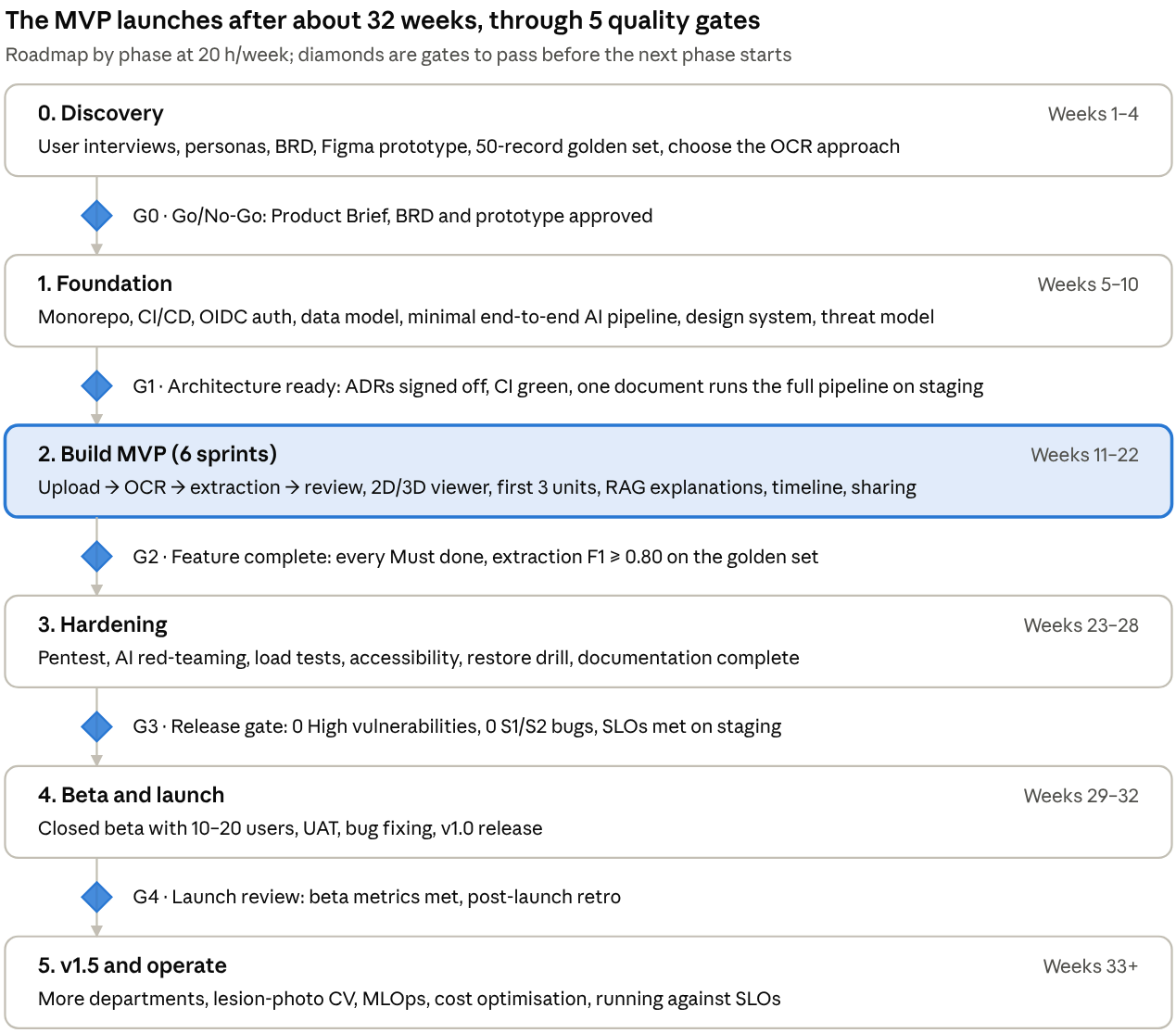
4.4 Sharing, storage and data rights

Code	Rule
BR-030	Sharing has a scope (all / per Unit / per document), an expiry (default 7 days, max 90) and can be revoked instantly.
BR-031	Share links work only for signed-in recipients; no anonymous public links.
BR-032	Users can export all their data (JSON/PDF) and request account deletion. Soft delete immediately, hard delete after 30 days (except audit logs kept as required).
BR-033	Original files are stored encrypted; shared previews carry a watermark with the recipient's name.
BR-034	User data is **not** used to train/fine-tune models without separate opt-in consent.
BR-035	Every read/write of medical data creates an AuditEvent; audit logs are append-only and kept at least 3 years (configurable).

5. Overall roadmap

The MVP (v1.0) launches after about **32 weeks** part-time; each phase ends at a gate with clear criteria — no gate, no next phase.

Update: the full 52-week / 26-sprint roadmap (adding hypercare, v1.5, v2.0 and the new gates) is in section 29. The v1.0 launch in week 32 is unchanged.



roadmap · 6 phases, 5 quality gates

Build is the heaviest phase (12 weeks, 6 sprints); if it slips, cut a Unit or the timeline (Should) rather than skipping Hardening.

5.1 Six sprints of the Build MVP phase

Sprint	Sprint goal	Demoable result
S1 (weeks 11–12)	Secure upload + state machine + OCR	Upload a photo, see OCR text and live status
S2 (weeks 13–14)	LLM extraction + Review screen	Correct and confirm results with bounding-box highlights
S3 (weeks 15–16)	Full-body 2D viewer + Dental unit	FDI tooth chart coloured from the record
S4 (weeks 17–18)	3D viewer + Cardiology unit	Rotate/zoom the model, click a region for details
S5 (weeks 19–20)	Oncology unit + RAG explanations	Tumour, nodes, metastases; Q&A with citations
S6 (weeks 21–22)	Pre/post timeline + sharing + admin	Compare two visits, time-limited sharing with a doctor

5.2 Sprint rituals (kept even when solo)

- **Day 1:** 1-hour Sprint Planning — pick stories from the backlog, write the Sprint Goal.
- **Daily:** 5-minute daily note (yesterday / today / blockers) in Notion.
- **Mid-sprint:** 1-hour backlog refinement for the next sprint (BA hat).
- **Last day:** Sprint Review (record a 5-minute demo video) + Retro (keep / drop / try) + update the risk register and Evaluation Report.

6. Plan per role

You play 12 roles. Each has a goal, work per phase, mandatory deliverables, tools, its own risks and skills to learn. Solo working rule: **2-week sprints**, opened with Sprint Planning (PO/PM hat) and closed with Review + Retro; every change goes through a Pull Request that you review with another role's checklist (e.g. backend code reviewed with the Security checklist).

Suggested time split (20 h/week)

Phase	BA/PO	UX/UI	AI/ML	DE	BE	FE	DevOps/SRE	Security	QA/QC
Discovery	50%	30%	15%					5%	
Foundation	10%	10%	15%	10%	20%	10%	15%	5%	5%
Build MVP	5%	10%	25%	10%	20%	20%	5%	5%	10%
Hardening	5%		10%	5%	10%	10%	15%	20%	25%
Launch & Operate	10%	5%	15%	10%	10%	10%	20%	10%	10%

6.1 Business Analyst (BA)

Goal: turn the idea into clear, testable, traceable requirements.

- **Discovery:** user interviews, As-Is/To-Be process maps (BPMN), stakeholders (RACI), BRD.
- **Foundation:** SRS/PRD: functional requirements (FR-xxx), non-functional requirements (NFR-xxx: performance, security, availability, accessibility), business rules (section 4), VI–EN medical glossary.
- **Build:** INVEST user stories, Gherkin acceptance criteria (Given/When/Then), use cases for complex flows (upload → review → confirm), clarify requirements when Dev asks (log questions in the backlog).
- **Hardening/Launch:** UAT scripts, traceability check (BR → Story → Test → Release), change request log.
- **Deliverables:** BRD, SRS/PRD, Use Case Spec, User Story Map, Data Dictionary, Glossary, Traceability Matrix, UAT Plan.
- **Tools:** Notion/Confluence, Jira or Linear, draw.io/Miro, Figma (read), Google Sheets.
- **Risks:** scope creep (every department at once); vague medical requirements; stories too big. **Mitigation:** lock MVP scope with MoSCoW, stories ≤ 3 days.
- **Learn:** BABOK basics, BPMN 2.0, Gherkin, ICD-10/SNOMED/LOINC/FHIR concepts.

Example user story:

```
US-021 – Confirm extraction results
As a patient, I want to see the original and the AI result side by side so I
can fix mistakes before saving.
Given I have a document in status "extracted"
When I open the Review screen
Then I see the original image with the text region of each field highlighted
And fields with confidence < 0.7 are yellow and must be confirmed
And "Save" is enabled only when every yellow field is confirmed (BR-012)
```

6.2 Product Owner / Project Manager (PO/PM)

Goal: decide what comes first, keep the schedule, manage risk and overall quality.

- **Discovery:** Product Vision, North Star Metric (e.g. *confirmed records viewed on the viewer per week*), quarterly OKRs, roadmap.
- **Ongoing:** product backlog (prioritised with RICE or WSJF), Sprint Planning/Review/Retro, burndown chart, risk register (section 11), release Go/No-Go.
- **Project management:** WBS, phase Gantt, milestones, Definition of Ready / Done, weekly status for yourself (Done / Next / Blockers / Risks).
- **Deliverables:** Product Vision, Roadmap, OKRs, Release Plan, Sprint Reports, Risk Register, Decision Log, Release Notes, Post-mortems.
- **Risks:** solo work tempts you to skip process; bad estimates. **Mitigation:** fixed timeboxes, 20% buffer per phase, written retros.
- **Learn:** Scrum Guide, Kanban, estimation (story points, T-shirt sizes), PMBOK risk management.

6.3 UX/UI Designer

Goal: someone with no medical background understands their condition in < 2 minutes without panicking.

- **Discovery:** user research, personas, journey map, information architecture (IA), sitemap.
- **Foundation:** low-fi wireframes: Onboarding/Consent, Dashboard, Upload, Review, Body Viewer (3D/2D), Unit Viewers (teeth, heart, tumour), Timeline, Explain panel, Share, Settings, Admin.
- **Build: Design System** (colour, typography, spacing, components, tokens), hi-fi mockups, prototype, severity colours (BR-020), loading/empty/error states, dark mode.

- **Healthcare specifics:** reassuring language, no shocking imagery (stylised models, not gory realism), disclaimers everywhere, WCAG 2.2 AA (contrast, keyboard, screen reader alternative to the 3D viewer as a region list).
- **Testing:** 5 users per usability round; measure task success and SUS score.
- **Deliverables:** IA/Sitemap, Wireframes, Design System, Hi-fi UI, Prototype, Usability Test Report, Accessibility checklist.
- **Tools:** Figma, FigJam, Maze/Useberry, Stark.
- **Risks:** a beautiful but hard-to-use 3D viewer; information overload. **Mitigation:** progressive disclosure (summary → detail), a simple 2D mode always available.

6.4 AI Solution Architect / ML Engineer (OCR, CV, LLM)

Goal: a pipeline that turns record images into correct, structured data with confidence scores, measurable and improvable.

AI pipeline (in order):

1. **Image pre-processing (classic CV):** blur/darkness check, deskew, document detection + perspective transform, denoise, contrast — OpenCV.
2. **Page classification:** prescription / discharge summary / lab result / pathology / X-ray report… — start with zero-shot vision LLM, later train a small classifier (EfficientNet/LayoutLMv3) once data exists.
3. **Layout + OCR:** detect text regions, tables, signatures, stamps. Options to compare: PaddleOCR (good Vietnamese, runs locally), VietOCR (printed Vietnamese), Tesseract (baseline), cloud services (Google Document AI, Azure Document Intelligence, AWS Textract) and multimodal LLMs. Handwriting: vision LLM or cloud OCR, always flagged low confidence.
4. **Information extraction:** LLM with **structured output (JSON Schema)** per Unit `extraction_schema`: diagnosis, ICD-10 code, anatomical site, laterality, size, TNM stage, lab values, dates, facility. Each field returns value + quoted source span (bounding box) + confidence.
5. **Normalisation:** abbreviations → full terms (Vietnamese THA → hypertension, ĐTĐ → diabetes), map to ICD-10 / SNOMED CT / LOINC via dictionary + embedding search + LLM rerank.
6. **Anatomical mapping:** text location ("right upper lobe", "tooth 36") → `region_code` + mesh ID. Rules first (FDI tooth regex, location dictionary), LLM fallback.
7. **Validation:** basic medical rules (teeth 11–48 valid, no future dates, sex–organ consistency), aggregate confidence.

8. **Explanation (RAG):** user question + confirmed data → retrieve KnowledgeArticles (vector DB) → LLM answer with citations and guardrails (BR-022, BR-023).
9. **Advanced computer vision (v1.5+):** skin lesion segmentation on photos (U-Net / SAM), tooth numbering on panoramic X-rays (YOLO + public data) — illustration only, never diagnosis.

Work per phase:

- **Discovery:** model survey, first evaluation set (golden set) of ~50 labelled synthetic records, trial 3 OCR options, ADR for the choice.
- **Foundation:** minimal end-to-end pipeline (notebook → service), JSON Schemas, prompt v1, automatic evaluation harness.
- **Build:** improve step by step against metrics; add normalisation, mapping, RAG, guardrails; prompt versioning; result caching.
- **Hardening:** red-teaming (prompt injection inside record images, dangerous questions), hallucination measurement, bias checks (handwritten vs printed, old vs new phone cameras).
- **Operate:** drift monitoring (is the correction rate rising?), learning from corrections (with consent), prompt/model A/B tests.

Creating data from nothing:

- Synthetic records: LLM-generated content per Unit template → rendered to PDF/images with HTML templates mimicking Vietnamese forms → variants (blur, skew, folds, phone capture, stamps, simulated handwriting fonts).
- Public data: MIMIC-IV-Note / MTSamples (English clinical text — registration and terms apply), FUNSD/CORD (layout), dental X-ray sets and ISIC skin images for CV. Always read the licence first.
- Dictionaries: ICD-10 (Ministry of Health Vietnamese edition), lab test catalogue, your own medical abbreviation list.
- Labelling: Label Studio; write an annotation guideline and measure agreement (re-label 10% after 2 weeks to check your own consistency).
- **Deliverables:** Model/Prompt Card per component, Evaluation Report each sprint, Dataset Card, model-choice ADR, Guardrail policy, AI Risk Assessment.
- **Tools:** Python, OpenCV, PaddleOCR, Hugging Face, PyTorch, LLM APIs (Claude/GPT/Gemini) or open models (Qwen2.5-VL, Llama) via vLLM/Ollama, LangGraph or plain code, pgvector, MLflow or Langfuse, Label Studio.

- **Risks:** poor handwriting OCR; LLM hallucination; rising API cost; sending health data to third-party APIs. **Mitigation:** mandatory human review, bounding-box citations, token budgets, PII redaction before API calls, prefer vendors with no-training commitments and suitable data residency.
- **Learn:** OCR and document AI, prompt engineering, structured output, RAG, LLM evaluation, MLOps basics.

6.5 Data Engineer (DE)

Goal: clean data with lineage, ready for analysis and retraining, with operational and analytical data clearly separated.

- **Foundation:** OLTP data model (PostgreSQL), object storage for originals, naming conventions, migrations (Alembic/Prisma), seed data for Department/Unit/Region/ICD-10 catalogues.
- **Build:** versioned ETL for standard catalogues (ICD-10, LOINC subset, abbreviation dictionary); synthetic data pipeline; KnowledgeArticle → chunk → embedding → pgvector pipeline.
- **Analytics:** a separate analytics schema (anonymised/pseudonymised): fact_extraction_run, fact_user_correction, dim_unit, dim_model_version… → AI quality and product dashboards. Tools: dbt + DuckDB/Postgres, Metabase or Superset.
- **Data quality:** Great Expectations / dbt tests: required fields not null, valid ICD codes and dates, confidence distribution not abnormally skewed.
- **Governance:** simple data catalogue (Data Dictionary), data classification (Public / Internal / Confidential / Sensitive-Health), retention and deletion policy, lineage original file → extraction → confirmed.
- **Deliverables:** ERD, Data Dictionary, ETL/ELT DAGs, Data Quality Report, Retention Policy, analytics dashboard.
- **Tools:** PostgreSQL 16 + pgvector, MinIO/S3, Prefect or Dagster, dbt, DuckDB, Great Expectations, Metabase.
- **Risks:** real data leaking into dev; schema changes breaking pipelines. **Mitigation:** synthetic-only dev/staging, schema contracts + migration tests in CI.

6.6 Backend Developer (BE)

Goal: a secure, consistent API that is easy to extend with new Units; heavy work runs asynchronously.

- **Architecture: modular monolith** (one codebase, clear modules) + a separate **AI worker**. Modules: `identity`, `patient`, `document`, `extraction`, `anatomy` (Department/Unit/Region config), `clinical` (Condition/Finding/Observation), `knowledge`, `sharing`, `notification`, `audit`, `admin`. Why: one person cannot run microservices well; a modular monolith still teaches domain boundaries (DDD).
- **Foundation:** project skeleton, per-environment config (12-factor), structured logging (JSON + correlation ID), standard errors (RFC 7807 Problem Details), OpenAPI written first (API-first), `/healthz` and `/readyz`.
- **Auth:** OAuth2/OIDC via Keycloak (self-hosted) or Auth0/Clerk/Supabase Auth; short-lived JWT + rotating refresh token; RBAC + ownership check on every query (PostgreSQL row-level security as a second layer).
- **Upload:** presigned URL to object storage → callback → job → worker → status updates via WebSocket/SSE. Virus scan (ClamAV), real MIME check, size limits.
- **Document state machine:** `uploaded → validating → rejected | queued → ocr_running → extracting → extracted → in_review → confirmed | failed`. Each transition is an event, with exponential-backoff retry and a dead-letter queue.
- **Main API (REST, `/api/v1`):** auth, `/me`, `/profiles`, `/documents` (upload, status, pages), `/extractions/{id}` (get, patch, confirm), `/conditions`, `/findings`, `/observations`, `/timeline`, `/units` and `/units/{code}/config`, `/explain` (stream), `/shares`, `/consents`, `/admin/*`, `/audit`.
- **Quality:** unit tests (pytest/Jest), integration tests with Testcontainers (real Postgres, Redis, MinIO in Docker), contract tests with FE (schemathesis or Pact), idempotency keys for important POSTs, cursor pagination, rate limiting.
- **Deliverables:** OpenAPI spec, ERD, sequence diagrams of main flows, ADRs (framework, auth, queue), one-command local README, coverage report.
- **Risks:** data leaks from missing ownership checks (IDOR); stuck jobs; N+1 queries. **Mitigation:** automated authorisation tests on every endpoint, timeouts + retries + DLQ, slow-query monitoring.

6.7 Frontend Developer (FE) — 3D/2D viewer

Goal: a smooth viewer (≥ 50 FPS on an average laptop), easy to understand, good on mobile web, accessible.

- **Base:** Next.js (App Router) + TypeScript, Tailwind + shadcn/ui per the Design System, TanStack Query (server state), Zustand (UI state), React Hook Form + Zod, VI/EN i18n (next-intl).

- **3D viewer:** Three.js via **React Three Fiber** + drei: OrbitControls (rotate, zoom, pan with mouse/touch), raycasting to pick regions, mesh highlighting by severity, transparent (x-ray) mode to see organs, camera presets (front, back, left, right), 3D labels. glTF/GLB models compressed with Draco/Meshopt, each mesh named by `region_code`.
- **3D model sources:** Z-Anatomy / BodyParts3D (open anatomy — check CC BY-SA licence), Sketchfab (suitably licensed), or your own simplified models in Blender. Learn basic Blender to split meshes, rename and decimate.
- **2D viewer:** SVG with an `id` per region (full body front/back, FDI tooth chart, heart, lungs, kidneys, GI tract). Fill colours, tooltips, click → detail panel. 2D is the default on mobile and the fallback without WebGL.
- **Unit viewers as plugins:** each Unit registers `{unitCode, Viewer2D, Viewer3D, legend, detailPanel}`; data comes from `/units/{code}/config` → new Units without core changes.
- **Pre/post timeline:** time slider, side-by-side compare (before | after), lab value charts over time (Recharts/ECharts), treatment event markers.
- **Review screen:** original image (zoom, pan) + bounding-box overlay, extraction form alongside, click a field → jump to its text region.
- **Performance:** lazy-load 3D models, LOD, route code-splitting, images via CDN, Lighthouse ⩾ 90 on non-viewer pages.
- **Testing:** Vitest + Testing Library, Playwright (E2E, screenshot visual regression for viewers), axe-core, Storybook.
- **Deliverables:** Storybook, normalised 3D/2D asset set, E2E suite, Lighthouse and accessibility reports.
- **Risks:** heavy 3D models; wrong mesh mapping; WebGL issues on weak devices. **Mitigation:** size budget (⩽ 5 MB per compressed model), automated mapping test (every `region_code` in config has a mesh), 2D fallback.

6.8 DevOps / Platform Engineer

Goal: commit to production automatically and repeatably, with rollback in < 10 minutes.

- **Environments:** `local` (Docker Compose) → `dev` → `staging` → `production`. Staging mirrors production but holds only synthetic data.
- **Source control:** GitHub, trunk-based development with short branches, Conventional Commits, branch protection (PR + green CI required), CODEOWNERS, semantic versioning + automated changelog.

- **CI (GitHub Actions):** lint → type check → unit tests → build → integration tests → SAST (Semgrep, CodeQL) → dependency scan (Dependabot/Trivy) → secret scan (gitleaks) → build image → scan image → push to registry.
- **CD:** auto-deploy to staging; production behind manual approval; blue-green or rolling; DB migrations as a separate, backward-compatible step (expand → migrate → contract).
- **Infrastructure:** Terraform IaC. Learning path: (1) one VPS + Docker Compose + Caddy/Traefik; (2) managed cloud (AWS: ECS Fargate/App Runner, RDS, S3, SQS or GCP: Cloud Run, Cloud SQL, GCS, Pub/Sub); (3) optional Kubernetes (k3s/EKS/GKE) + Helm + Argo CD for GitOps.
- **Secrets:** never in the repo; GitHub Secrets + AWS Secrets Manager/GCP Secret Manager or Doppler/Infisical.
- **Deliverables:** infra diagram, Terraform modules, pipeline YAML, deploy/rollback runbook, optional per-PR ephemeral environments.
- **Risks:** cloud cost overrun; environment drift. **Mitigation:** budget alerts, staging off out of hours, everything via IaC.

6.9 Security & Compliance Engineer

Goal: health data protected by privacy-by-design; compliant with the Law on Personal Data Protection 91/2025/QH15 (and its guiding Decree 356/2025/ND-CP), referencing HIPAA/GDPR and OWASP.

- **Discovery:** data classification, Data Protection Impact Assessment (DPIA), lawful basis (consent), **STRIDE** threat model on the data flow diagram (DFD).
- **Foundation — Secure SDLC:** security requirements (ASVS level 2), secure coding guideline, dependency policy, secret management, SAST/DAST/SCA in CI.
- **Technical controls:**
  - Encryption: TLS 1.2+ everywhere, HSTS; AES-256 at rest (KMS); application-level encryption of sensitive fields (envelope encryption).
  - Identity: MFA, NIST 800-63B password policy, breached-password check (HIBP k-anonymity), secure sessions (HttpOnly, Secure, SameSite cookies).
  - Application: OWASP Top 10 (IDOR, injection, XSS, CSRF, SSRF), strict CSP, schema input validation, upload limits, malware scanning.
  - AI: OWASP Top 10 for LLM — prompt injection (including hidden text in record images), data leakage via prompts, insecure output; PII redaction before external LLMs; redacted prompt logs.

  - Infrastructure: least-privilege IAM, network segmentation (no public DB), WAF, encrypted backups, CIS benchmarks.
  - Monitoring: immutable audit log, anomaly alerts (logins from unusual countries, bulk downloads).
- **Hardening:** self-run pentest with OWASP ZAP and Burp Suite Community against the ASVS checklist; fix by SLA (Critical 24h, High 7 days, Medium 30 days).
- **Compliance docs:** Privacy Policy, Terms of Service, consent text, DPIA, Records of Processing, Incident Response Plan (breach notification within 72 hours of detection per Decree 356/2025), Data Retention Policy, vendor assessment (LLM/cloud providers).
- **Risks:** a health-data leak is the project's most severe risk. **Mitigation:** synthetic data only until the security checklist passes; for a public demo, block real uploads or invite-only access.

6.10 QA Engineer (Quality Assurance — preventing defects)

Goal: quality built into the process, not inspected at the end.

- **Test Strategy & Test Plan:** test pyramid (many unit, some integration, few E2E), test environments, test data, entry/exit criteria, defect management (severity S1–S4, priority P1–P4).
- **Test types:** functional, regression, API/contract, E2E, cross-browser (Chrome, Safari, Firefox, mobile Safari/Chrome), performance (k6: 100 concurrent users, API p95 < 300 ms), AI pipeline load (100 documents/hour), security, accessibility, usability, **AI evaluation** (golden set on every PR that changes a prompt/model).
- **Test cases from acceptance criteria:** every BR and US has at least one test; boundary values, equivalence partitioning, decision tables (sharing permissions), state transitions (document state machine).
- **Shift-left:** review requirements for testability before coding; tests in the Definition of Done.
- **Deliverables:** Test Strategy, Test Plan, Test Cases (TestRail/Qase or Markdown in the repo), Bug Reports, Test Summary Report per release, automation suite.

6.11 QC (Quality Control — checking the product before release)

- **Release checklist:** all tests green, no open S1/S2 bugs, AI metrics not below the previous release, migrations rehearsed on a staging copy, complete release notes.

- **Medical content check:** every KnowledgeArticle and new Unit mapping reviewed against a checklist (sources, date, plain language, not alarming, disclaimer).
- **Exploratory testing:** 2 hours per sprint against a charter (e.g. "break the Review screen with bad photos, non-record images, a 30-page record").
- **UAT:** you run UAT scripts as Patient/Clinician; invite 3–5 real people in beta.
- **Quality measures:** defect density, defect leakage, escaped defects by severity.

6.12 SRE / Maintenance & Support

Goal: a stable system, incidents detected before users notice, fast recovery.

- **Observability:** logs (Loki or CloudWatch), metrics (Prometheus + Grafana), traces (OpenTelemetry → Tempo/Jaeger), error tracking (Sentry), uptime checks (Uptime Kuma / Better Stack). LLM observability: Langfuse (tokens, cost, latency, quality).
- **SLI/SLO:** API availability 99.5%/month; API p95 < 300 ms; 95% of documents processed in < 2 minutes; error budget with a feature-freeze policy when exceeded.
- **Incident management:** SEV1–SEV4, on-call (you) with Telegram/Slack alerts, a runbook per alert, blameless post-mortem within 48 hours.
- **Backup & DR:** daily DB backups + PITR, versioned object storage; **RPO 24h, RTO 4h**; quarterly restore drill into a temporary environment.
- **Maintenance:** weekly dependency updates (Renovate), monthly OS/image patching, monthly cost review, retention cleanup, refresh expired medical content, re-evaluate when providers release new models.
- **Support:** in-app feedback, FAQ, ticket triage (bug / question / data deletion request), 2-business-day response SLA.
- **Deliverables:** SLO doc, dashboards, alert rules, runbooks, DR plan + drill records, post-mortems, maintenance calendar.

7. Tech stack and rationale

Selection principle: **widely used in industry (career value), well documented, operable by one person, cheap with zero users.** Every choice is recorded as an ADR (Architecture Decision Record) listing the options considered.

Layer	Main choice	Alternatives	Why
Frontend	Next.js 15 + React + TypeScript	Vite + React SPA, Nuxt	SSR for public pages, routing, huge ecosystem
UI	Tailwind CSS + shadcn/ui + Radix	MUI, Ant Design	Accessible primitives, easy custom Design System
3D	Three.js + React Three Fiber + drei	Babylon.js	Largest community, fits React
2D	SVG + D3 (interaction)	Konva, Canvas	Per-region ids, easy colouring and accessibility
Charts	ECharts or Recharts	Chart.js	Lab value timelines
Backend API	Python FastAPI	NestJS (Node), Spring Boot	Same language as AI, async, auto OpenAPI
ORM / migrations	SQLAlchemy 2 + Alembic	Prisma (with NestJS)	Python industry standard
AI worker	Python + Celery or Dramatiq (Redis)	Temporal, AWS Step Functions	Async jobs with retries; Temporal to learn later
OCR	PaddleOCR + VietOCR (local), compared with Google Document AI / Azure Document Intelligence	Tesseract, AWS Textract	Local for sensitive data; cloud as accuracy baseline
LLM	APIs with structured output (Claude, GPT, Gemini) behind an abstraction layer	Self-hosted Qwen2.5-VL / Llama (vLLM, Ollama)	Fast start via API; switch providers without code changes

Layer	Main choice	Alternatives	Why
CV	OpenCV, PyTorch, Ultralytics YOLO, SAM	TensorFlow	Image pre-processing, tooth numbering, skin segmentation
RAG	pgvector + multilingual embeddings (bge-m3 / multilingual-e5)	Qdrant, Weaviate	Fewer moving parts, same DB
Database	PostgreSQL 16	MySQL	JSONB, RLS, pgvector, ubiquitous
Cache / queue	Redis 7	RabbitMQ, SQS	Cache, rate limiting, broker
Object storage	S3 (prod) / MinIO (local)	GCS, Azure Blob	Originals, encryption, presigned URLs
Auth	Keycloak (OIDC)	Auth0, Clerk, Supabase Auth	Learn full OIDC; self-hosted; MFA, RBAC
Realtime	Server-Sent Events / WebSocket	Pusher, Ably	Processing status updates
Data orchestration	Prefect or Dagster + dbt	Airflow	Lighter than Airflow for one person
MLOps / LLMOps	MLflow + Langfuse	Weights & Biases	Experiments, prompts, evals, cost
Labelling	Label Studio	CVAT	Images + text
Containers	Docker, Docker Compose	Podman	Identical environments
IaC	Terraform	Pulumi	Enterprise standard
Cloud	AWS (ECS Fargate, RDS, S3, SQS, KMS, CloudFront)	GCP (Cloud Run…), VPS + Coolify	Most common; a VPS works early on
CI/CD	GitHub Actions	GitLab CI	Built into the repo

Layer	Main choice	Alternatives	Why
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Sentry	Datadog, New Relic	Open source, learn the fundamentals
Security tools	Semgrep, CodeQL, Trivy, gitleaks, OWASP ZAP, Renovate	Snyk, SonarQube	Free for personal projects
Testing	pytest, Vitest, Testing Library, Playwright, k6, Testcontainers, Storybook	Cypress, JMeter	Covers every layer
Project management	Linear or Jira, Notion/Confluence, Figma, draw.io	GitHub Projects	Simulates an enterprise setup

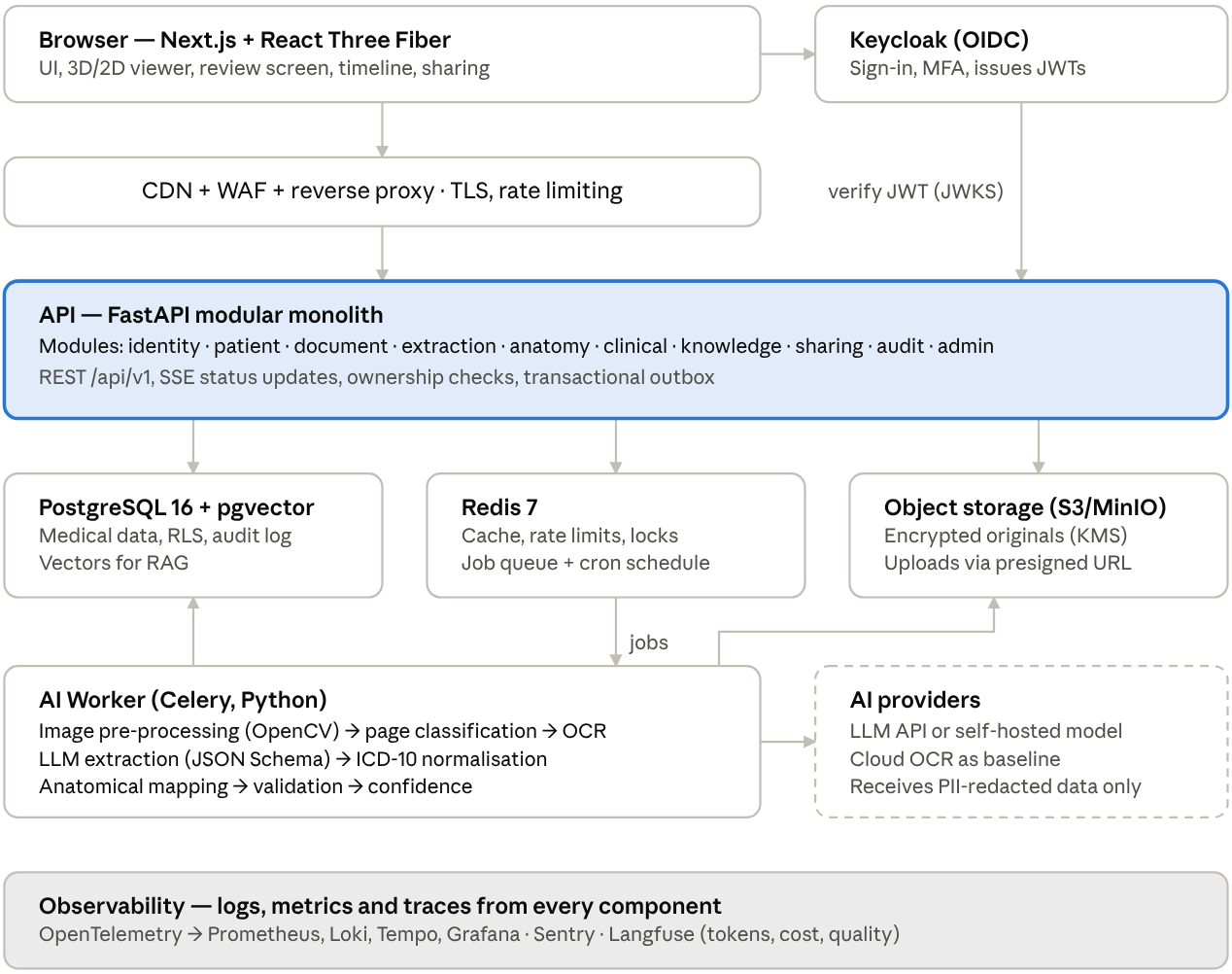
Repository layout (monorepo):

```
medilens/
├── apps/
│   ├── web/              # Next.js frontend
│   ├── api/              # FastAPI modular monolith
│   └── worker/           # AI pipeline worker
├── packages/
│   ├── anatomy-assets/   # GLB, SVG, unit configs
│   ├── api-client/       # TS client generated from OpenAPI
│   └── design-system/
├── ml/
│   ├── datasets/         # synthetic data scripts (never commit real data)
│   ├── eval/             # golden set + evaluation harness
│   └── notebooks/
├── data/                 # dbt, Prefect flows
├── infra/                # Terraform, Helm, docker-compose
├── docs/                 # BRD, SRS, ADRs, runbooks, threat model
└── .github/workflows/
```

8. System architecture and data model

The architecture is one **modular-monolith API** for synchronous requests and one **AI worker** that processes documents asynchronously through a queue — enough boundaries to learn enterprise architecture, simple enough for one person to run.

One modular-monolith API and one AI worker, linked through a Redis queue



system architecture · API, worker, 3 data stores

The worker never takes user requests directly; it pulls jobs from Redis, reads files from object storage and writes results to PostgreSQL. Add full **C4 model** diagrams (Context → Container → Component) in `docs/architecture`.

8.1 Processing one document

1. FE requests a presigned URL → uploads straight to object storage (files never pass through the API).
2. FE calls `POST /documents/{id}/complete` → API writes `DocumentUploaded` to the outbox.
3. The worker runs the job chain in section 9.1 and updates status; FE receives it via SSE.
4. The user reviews and confirms → `clinical.materialize` creates Condition/BodyFinding/Observation.
5. FE calls `/units/{code}/config` + `/findings` → the viewer colours regions by `region_code`.

8.2 Data model (main tables)

Table	Key columns	Notes
`users`	id (UUID), email, status, created_at	Synced from Keycloak via `sub`
`patient_profiles`	id, owner_user_id, display_name, birth_year, sex_at_birth, is_minor	One user manages several profiles
`documents`	id, profile_id, storage_key, sha256, mime, pages, status, doc_type, uploaded_at	`sha256` for duplicates (BR-016)
`document_pages`	id, document_id, page_no, ocr_text, ocr_boxes (JSONB), quality_score	Bounding boxes for the review screen
`extraction_runs`	id, document_id, pipeline_version, model, prompt_version, raw_output (JSONB), confidence, cost_usd, status	Traces every AI run
`extraction_fields`	id, run_id, field_path, value, confidence, source_page, source_box, user_value, confirmed_at	Keeps both AI and user-corrected values

Table	Key columns	Notes
`encounters`	id, profile_id, date, facility, department_code	One visit
`conditions`	id, profile_id, encounter_id, icd10_code, unit_code, status, severity, stage, onset_date	Status over time
`body_findings`	id, condition_id, region_code, laterality, size_mm, severity, is_metastasis, observed_at	What the viewer draws
`observations`	id, profile_id, loinc_code, name, value, unit, ref_low, ref_high, observed_at	Lab charts
`departments`, `units`, `regions`	code, name_vi, name_en, parent_code, mesh_id, svg_id, std_code	Versioned catalogues
`unit_configs`	unit_code, version, config (JSONB), published_at	Config from section 3.2
`knowledge_articles`, `knowledge_chunks`	id, condition/region refs, body, sources, reviewer, reviewed_at, embedding (vector)	RAG
`share_grants`	id, profile_id, grantee_user_id, scope, expires_at, revoked_at	BR-030, BR-031
`consents`	id, user_id, type, terms_version, given_at, withdrawn_at	BR-002, BR-034
`audit_events`	id, actor_id, action, resource_type, resource_id, ip, user_agent, at	Append-only, partitioned monthly
`outbox`	id, event_type, payload, created_at, published_at	Transactional outbox

Conventions: UUIDv7 primary keys, `created_at/updated_at` on every table, soft delete via `deleted_at`, Row-Level Security by `profile_id`, application-level encryption for sensitive fields (name, date of birth).

9. Cron, background jobs and events

Three kinds: **event-driven jobs** (run on an action), **scheduled crons** (run periodically) and **internal domain events** (one module notifies another). Every job must be idempotent, have a timeout and bounded retries, log with a correlation ID, and land in a dead-letter queue on final failure.

9.1 Event-driven jobs (queue)

Job	Trigger	Work	Timeout / Retries
`document.validate`	Upload completed	MIME, virus, size, blur check, split PDF pages	60 s / 3
`document.preprocess`	Validation OK	Deskew, crop, denoise, thumbnails	120 s / 3
`document.classify`	Pre-processing done	Page type classification	60 s / 3
`document.ocr`	Classification done	OCR per page, store text + bounding boxes	300 s / 3
`extraction.run`	OCR done	LLM extraction per Unit schema, code normalisation, region mapping	300 s / 2
`extraction.validate`	Extraction done	Medical rules, confidence, flag fields for review	30 s / 3
`clinical.materialize`	User confirms	Create/update Condition, BodyFinding, Observation; recompute timeline	60 s / 5
`notification.send`	Many events	Email/in-app (processing done, new share, share expiring)	30 s / 5
`export.generate`	User request	Build JSON/PDF data package, 24h download link	600 s / 2
`account.delete`	Deletion request	Soft delete now; schedule hard delete after 30 days	300 s / 5

Job	Trigger	Work	Timeout / Retries
`knowledge.embed`	Article approved	Chunk + embed + upsert vectors	120 s / 3

9.2 Scheduled crons (Vietnam time, UTC+7)

Cron	Schedule	Work
`share.expire`	Every 15 minutes	Disable expired ShareGrants, write audit
`job.reaper`	Every 10 minutes	Find jobs stuck past timeout → mark failed, retry or DLQ
`upload.cleanup`	Hourly	Delete abandoned uploads/unused presigned URLs after 24h
`ai.eval.nightly`	02:00 daily	Run the golden set on the current model/prompt, compare to baseline, alert if down > 3 points
`analytics.etl`	03:00 daily	dbt run + test: refresh anonymised analytics tables
`backup.verify`	04:00 daily	Check last night's backup exists, sane size, checksum
`account.purge`	05:00 daily	Hard-delete accounts soft-deleted > 30 days ago
`reminder.followup`	08:00 daily	Follow-up appointment reminders set by users
`cost.report`	09:00 Monday	Last week's cloud + LLM token cost, budget alerts
`deps.audit`	10:00 Monday	Scan dependency/image CVEs, open issues
`knowledge.review_due`	1st of month	List KnowledgeArticles not re-reviewed for 12 months (BR-024)
`retention.enforce`	1st of month	Apply retention to logs, temp files, analytics data

Cron	Schedule	Work
`dr.drill`	Quarterly	Restore drill reminder and checklist
`access.review`	Quarterly	Review Admin/Clinician accounts, lock inactive ones

9.3 Internal domain events

`UserRegistered`, `ConsentGiven`, `ConsentWithdrawn`, `DocumentUploaded`, `DocumentRejected`, `ExtractionCompleted`, `ExtractionConfirmed`, `FindingCreated`, `ShareGranted`, `ShareRevoked`, `AccountDeletionRequested`, `KnowledgeArticleApproved`.

Use the **transactional outbox pattern**: write the event to an `outbox` table in the same transaction as the data; a relay process reads the outbox and publishes to the queue. This key enterprise pattern prevents lost events when the system fails mid-way.

Operational notes: crons run on Celery Beat (local/VPS) or EventBridge Scheduler (AWS); only one instance runs each cron (Redis distributed lock); every cron exposes a "last successful run" metric with an alert when overdue (dead man's switch).

10. Metrics for evaluating the app

Measure on 4 layers: **AI**, **product**, **engineering/operations**, **process quality**. Each metric has a definition, a method, an MVP threshold and a target. AI metrics use a **fixed golden set** (never used for prompt tuning) plus a **dev set** (used for tuning) to avoid overfitting.

10.1 AI metrics (per Unit and per document type)

Component	Metric	Method	MVP	Target
Image quality check	Precision/Recall when rejecting bad images	Against manual good/bad labels	⩾ 0.85	⩾ 0.95
Page classification	Accuracy, macro-F1	Confusion matrix	⩾ 0.85	⩾ 0.95
Printed OCR	CER (Character Error Rate), WER	Text vs ground truth	CER ⩽ 5%	CER ⩽ 2%

Component	Metric	Method	MVP	Target
Handwritten OCR	CER	Same	⩽ 20%	⩽ 10%
Field extraction	Field-level Precision / Recall / F1 (exact & fuzzy)	JSON vs labels	F1 ⩾ 0.80	F1 ⩾ 0.92
ICD-10 normalisation	Top-1 / Top-3 accuracy	Against correct code	Top-1 ⩾ 0.75	⩾ 0.90
Anatomical mapping	Region accuracy (right region + side)	Against labels	⩾ 0.85	⩾ 0.95
Tooth number (FDI)	Tooth-level accuracy	Against labels	⩾ 0.90	⩾ 0.98
Confidence	Calibration (ECE), error rate among conf ⩾ 0.7	Reliability diagram	ECE ⩽ 0.1	⩽ 0.05
RAG explanations	Faithfulness, answer relevance, context precision	RAGAS / LLM-as-judge + 50 hand-graded samples	⩾ 0.85	⩾ 0.95
Hallucination	Share of unsourced claims	Manual + judge	⩽ 5%	⩽ 1%
Guardrails	Correct block rate on dangerous questions / false block rate	100-question red-team set	⩾ 95% / ⩽ 10%	⩾ 99% / ⩽ 5%
CV segmentation (v1.5)	Dice / IoU	Against label masks	Dice ⩾ 0.75	⩾ 0.85
CV tooth detection (v1.5)	mAP@0.5	COCO eval	⩾ 0.80	⩾ 0.90
Pipeline performance	Processing time per document (p50, p95)	Tracing	p95 ⩽ 3 min	⩽ 60 s
Cost	AI cost per document	Tokens + compute	⩽ $0.10	⩽ $0.03

Production metrics (no labels): field correction rate, document rejection rate, "unmapped" rate, confidence distribution over time (drift detection).

10.2 Product metrics

Metric	Definition	Beta target
North Star	Confirmed records viewed on the viewer / week	+10%/week during beta
Activation rate	% of sign-ups who upload and confirm ⩾ 1 document within 7 days	⩾ 40%
Time-to-first-insight	Time from upload to first viewer visit	⩽ 5 min
Review completion	% of extracted documents confirmed by users	⩾ 70%
Retention	D7 / D30	⩾ 25% / ⩾ 15%
Comprehension score	After viewing, user answers 3 questions about their condition correctly	⩾ 80%
SUS / CSAT	Usability / satisfaction score	SUS ⩾ 70, CSAT ⩾ 4/5
Task success	% of testers completing upload → viewer unaided	⩾ 80%

10.3 Engineering and operations metrics

- **DORA:** deployment frequency (⩾ 1/week), lead time for changes (⩽ 1 day), change failure rate (⩽ 15%), MTTR (⩽ 4 hours).
- **SLOs:** availability ⩾ 99.5%, API p95 ⩽ 300 ms, p99 ⩽ 1 s, 5xx error rate ⩽ 0.5%.
- **Frontend:** LCP ⩽ 2.5 s, INP ⩽ 200 ms, CLS ⩽ 0.1; 3D viewer ⩾ 50 FPS (desktop), ⩾ 30 FPS (mobile); model load ⩽ 3 s on 4G.
- **Cost:** monthly infrastructure cost and cost per active user.

10.4 Quality and security metrics

- Test coverage: BE ⩾ 80%, FE ⩾ 70%, endpoint authorisation 100%.
- Defect leakage ⩽ 10%; no open S1 bugs at release.
- Vulnerabilities: 0 Critical/High open past SLA; mean time to patch.
- Accessibility: 0 serious/critical axe issues; WCAG 2.2 AA.

- Documentation: 100% of features have story + AC + tests; 100% of architecture decisions have ADRs.

Reporting: an **Evaluation Report** every sprint (AI metric table, delta vs last sprint, typical errors) and a **Quality Dashboard** in Grafana/Metabase. This is also the strongest portfolio piece for interviews.

11. Risk register by area

The biggest risks are not technical: **user safety (wrong medical information) and health-data leaks**, followed by **solo overload leading to abandonment**. Score = Likelihood (1–5) × Impact (1–5); ≥ 15 is red, reviewed every sprint.

ID	Area	Risk	L	I	Score	Mitigation	Contingency
R01	Medical / safety	Users mistake illustrations for diagnosis and self-treat	3	5	15	Disclaimers on every screen, "your record states…" wording, no prognosis, guardrails	Remove explanations, keep visualisation only
R02	Security	Health-data leak (IDOR, misconfigured S3, leaked secrets)	3	5	15	Threat model, authorisation tests, RLS, encryption, secret scanning, synthetic data until audited	Incident Response Plan, rotate keys, notify within 72h
R03	Project	One person takes on too much, burns out, abandons	4	4	16	3-Unit MVP, timeboxes, 20% buffer, retros, permission to cut scope	Freeze at the nearest milestone and publish as a case study
R04	AI	LLM hallucination	4	4	16	Structured output + bounding-box citations, human review, RAG on approved sources only	Lower auto thresholds, force review of every field
R05	AI	Poor OCR on handwriting and bad phone photos	5	3	15	Image quality check, capture guidance, cloud OCR fallback	Assisted manual entry
R06	Data	No real data to train/evaluate	5	3	15	Synthetic generator, public datasets, self-labelling	Limit MVP to printed records
R07	Legal	Breaching PDPL 91/2025 / Decree 356/2025 on sensitive data, cross-border transfer via APIs	3	5	15	Clear consent, DPIA, PII redaction, vendor choice, consider self-hosted models	Run demos on synthetic data only

ID	Area	Risk	L	I	Score	Mitigation	Contingency
R08	Content	Wrong or outdated explanations	3	4	12	Authoritative sources, review dates, re-review cron	Hide the article, show "ask your doctor"
R09	Licensing	3D models/SVGs/datasets violate licences	3	3	9	Record each asset's licence in `ATTRIBUTION.md`, CC/open only	Replace with simple self-made assets
R10	Frontend	Heavy, laggy 3D viewer on weak/mobile devices	4	3	12	Compression, LOD, size budget, 2D fallback	2D by default on mobile
R11	Mapping	Wrong region or side (left ↔ right)	3	4	12	Laterality rules, mapping tests, show source text on region click	Hide low-confidence regions (unmapped)
R12	Cost	LLM/cloud spend exceeds budget	3	3	9	Caching, small models for simple steps, budget alerts, daily upload caps	Move to self-hosted models
R13	Vendor	LLM provider changes pricing/retires a model, results shift	3	3	9	Provider abstraction, nightly eval, pinned model versions	Switch provider after passing the golden set
R14	Operations	Data loss from failed migration or broken backup	2	5	10	Expand/contract migrations, backup + verify, DR drills	PITR restore
R15	Engineering	Architecture too complex for one person (early microservices, K8s)	3	3	9	Modular monolith, K8s only as a post-MVP learning goal	Back to Docker Compose/managed services
R16	Ethics / UX	Metastasis visuals and cancer content cause fear and distress	3	4	12	Reassuring language, "summary only" mode, no prognosis figures, mental-health support links	Let users hide "possible spread"

Risks by role (summary): BA – scope creep, vague requirements · UX – information overload · AI – hallucination, drift, cost · DE – real data in dev, broken schemas · BE – IDOR, stuck jobs · FE – 3D performance, mesh mapping · DevOps – environment drift, cost · Security – leaks, prompt injection · QA/QC – false-green tests, unrealistic test data · SRE – noisy alerts, untested restores.

12. Resources, cost, required documents and Definition of Done

The project is feasible with **~1,000–1,200 hours** and **about $20–80/month in cash** using free tiers and synthetic data. Cost figures below are **rough estimates**; check current prices before committing.

12.1 People and time

Phase	Duration (20 h/week)	Hours
0. Discovery	4 weeks	80
1. Foundation	6 weeks	120
2. Build MVP (6 sprints)	12 weeks	240
3. Hardening	6 weeks	120
4. Beta & Launch	4 weeks	80
5. v1.5 expansion + Operate	12–16 weeks	240–320
Learning new skills (in parallel)	Whole project	150–200

Helpful but optional: a doctor or medical student to review content (1 hour/month), 5–8 usability testers, a technical mentor reviewing the architecture once per phase.

12.2 Hardware and cost (estimates)

Item	Budget option	Estimated cost/month
Dev machine	16 GB RAM laptop (32 GB for local models); GPU optional	Already owned
GPU (CV training, self-hosted models)	Free Google Colab / Kaggle, hourly GPU rental when needed	$0–20
Early hosting	One 4 vCPU / 8 GB VPS + Docker Compose	$10–25
Enterprise-learning cloud	AWS/GCP free tier + student/startup credits	$0–50

Item	Budget option	Estimated cost/month
LLM API	Monthly budget cap, caching	$5–30
Domain + transactional email	.com/.vn domain, Resend/SES	$1–5
Tools	GitHub, Linear, Notion, Figma, Sentry, Grafana Cloud — free plans	$0

12.3 Document set an enterprise project needs

Group	Documents	Owner role
Product	Product Vision, Product Brief, Roadmap, OKRs, Release Notes	PO/PM
Business	BRD, SRS/PRD, Use Cases, User Stories + AC, Business Rules, Glossary, Traceability Matrix	BA
Design	IA, Wireframes, Design System, Prototype, Usability Report	UX/UI
Architecture	Architecture Overview (C4 model), ADRs, ERD, Sequence diagrams, OpenAPI	BE / Architect
AI	Dataset Card, Model/Prompt Card, Evaluation Report, Guardrail Policy, AI Risk Assessment	AI/ML
Data	Data Dictionary, Lineage, Data Quality Report, Retention Policy	DE
Infrastructure	Infra diagram, IaC, CI/CD docs, Environment matrix	DevOps
Security & legal	Threat Model, DPIA, Security Requirements, Pentest Report, Privacy Policy, Terms, Incident Response Plan	Security
Quality	Test Strategy, Test Plan, Test Cases, Test Summary Report, UAT sign-off	QA/QC
Operations	SLOs, Runbooks, On-call guide, DR Plan, Post-mortems, Maintenance calendar	SRE

12.4 Definition of Ready

- [ ] INVEST user story with Gherkin acceptance criteria
- [ ] Linked business rules (BR-xxx) and UI design (if any)
- [ ] Estimated ⩽ 3 days; dependencies known
- [ ] Test data and security/privacy requirements identified

12.5 Definition of Done

- [ ] Code merged via PR, self-reviewed with the Security + QA checklists
- [ ] New unit/integration tests, CI green, coverage not lower
- [ ] Acceptance criteria pass (automated or recorded manual test)
- [ ] No High+ SAST/SCA findings
- [ ] Logging, metrics, error handling in place; access control tested
- [ ] Docs updated (OpenAPI, README, ADR if a new decision was made)
- [ ] Deployed to staging and smoke-tested
- [ ] For AI changes: golden set run, metrics down by no more than 1 point

12.6 Parallel learning path

1. **Months 1–2:** Scrum/BA basics, Figma, FastAPI, PostgreSQL, Docker, Git workflow.
2. **Months 3–4:** OpenCV, OCR, LLM structured output, React Three Fiber, basic Blender.
3. **Months 5–6:** RAG, LLM evaluation, OIDC/Keycloak, testing (Playwright, Testcontainers), GitHub Actions.
4. **Months 7–8:** Terraform + AWS, observability (OpenTelemetry, Grafana), OWASP ASVS, threat modelling.
5. **Month 9+:** MLOps, advanced CV (YOLO, SAM), Kubernetes/GitOps (optional), FHIR.

Part II – Enterprise-grade specifications

Part I answers *what to do and who does it*. Part II answers *what standard it must meet*: every requirement has a code, a strength (**MUST** / **SHOULD** / **MAY** per RFC 2119), a verification method and a phase. This is how large companies write specifications that can be audited, tested and handed over. Shared prefixes: `SEC` security, `DAT` data, `DE` data engineering, `NFR` non-functional, `BE` backend, `FE` frontend, `OPS` operations.

13. Detailed security requirements

The security target is **OWASP ASVS 5.0 Level 2** for the application, **OWASP Top 10 for LLM** for AI, **CIS AWS Foundations** for infrastructure, and compliance with Vietnam's **Law on Personal Data Protection No. 91/2025/QH15** (in force since 1 January 2026, guided by Decree 356/2025/ND-CP, which replaced Decree 13/2023) — health data is sensitive personal data. Every requirement below needs an automated test or evidence in the Security Evidence Pack before gate G3.

13.1 Defence in depth

1. **Identity** — who gets in (Cognito/Keycloak, MFA, sessions).
2. **Application** — what they may do (RBAC + ownership + RLS).
3. **Data** — how data is protected (encryption, masking, deletion).
4. **AI** — can the model be tricked or leak data (guardrails, redaction).
5. **Infrastructure** — network, IAM, secrets, images.
6. **Detection & response** — logs, alerts, incident response.
7. **Process** — Secure SDLC, reviews, pentests, training (self-study).

13.2 Identity and authentication

Code	Requirement	Strength	Verification
SEC-ID-01	OIDC Authorization Code + PKCE only; no Implicit flow	MUST	IdP config review, E2E
SEC-ID-02	Passwords ≥ 12 chars, breached-password check (HIBP k-anonymity), no forced periodic change (NIST 800-63B)	MUST	Unit test
SEC-ID-03	MFA (TOTP/WebAuthn) mandatory for Clinician and Admin; recommended for Patient, mandatory when sharing is enabled	MUST	E2E per role
SEC-ID-04	JWT access tokens ≤ 15 min, RS256/ES256; rotating refresh tokens with reuse detection revoking the chain	MUST	Integration test

Code	Requirement	Strength	Verification
SEC-ID-05	Session cookies `HttpOnly`, `Secure`, `SameSite`; no tokens in localStorage	MUST	Header scan, FE test
SEC-ID-06	Lockout after 5 failures, rate limits per IP + account, CAPTCHA after 3	MUST	k6 brute-force test
SEC-ID-07	Sign out all devices; active sessions list	SHOULD	E2E
SEC-ID-08	Email alert on new device/country, password/MFA change	SHOULD	E2E
SEC-ID-09	Clinician registration verified (licence upload, Admin approval)	SHOULD (v1.5)	Process + audit

13.3 Authorization

Code	Requirement	Strength	Verification
SEC-AZ-01	Deny by default: every endpoint requires authentication and an explicit policy	MUST	Automated route listing; route without policy fails CI
SEC-AZ-02	Ownership check in the service layer for **every** ID-based query (anti-IDOR/BOLA)	MUST	Matrix: user A requests user B's resource → 404
SEC-AZ-03	PostgreSQL Row-Level Security by `profile_id` as a second layer	MUST	SQL tests as the app role
SEC-AZ-04	Share scope + expiry checked at access time, cache ⩽ 60 s	MUST	Expiry/revocation integration tests
SEC-AZ-05	Admins cannot read PHI; break-glass needs a reason, lasts 1 h, alerts immediately, audited	MUST	E2E + alert check
SEC-AZ-06	Separation of duties: medical content authors don't approve their own articles (when ⩾ 2 reviewers)	SHOULD	Workflow test

Code	Requirement	Strength	Verification
SEC-AZ-07	Mass assignment prevented: DTO allowlists only	MUST	Contract test

13.4 Data protection

Code	Requirement	Strength	Verification
SEC-DP-01	TLS 1.2+ (prefer 1.3) everywhere including VPC traffic to RDS; HSTS 1 year, preload	MUST	SSL Labs A+, `rds.force_ssl=1`
SEC-DP-02	At-rest encryption with AWS KMS customer-managed keys for RDS, S3, EBS, backups, SQS; yearly rotation	MUST	AWS Config rule
SEC-DP-03	Application-level envelope encryption for identifiers: name, DOB, phone, national ID (if any)	MUST	Unit test + unreadable DB dump
SEC-DP-04	S3: account-level Block Public Access, TLS-only bucket policy, BucketOwnerEnforced, versioning + Object Lock for audit bucket	MUST	AWS Config, Access Analyzer
SEC-DP-05	Presigned upload/download URLs ⩽ 5 min with content-type and size limits	MUST	Integration test
SEC-DP-06	No PHI/PII in logs, traces or error tracking; redaction middleware + automated log scan	MUST	Automated log scan
SEC-DP-07	Dev/staging hold synthetic data only; copying prod snapshots down is blocked	MUST	SCP blocking cross-account snapshot sharing
SEC-DP-08	Deletion by crypto-shredding (delete the user's data key) so backups are covered	SHOULD	Runbook + test

Code	Requirement	Strength	Verification
SEC-DP-09	Dynamic watermark on shared images (disabling right-click is cosmetic only)	SHOULD	E2E

13.5 Application and API security

Code	Requirement	Strength	Verification
SEC-AP-01	Validate all input with schemas (Pydantic/Zod); reject unknown fields	MUST	Contract + fuzz (schemathesis)
SEC-AP-02	ORM/parameterised queries only; no SQL string concatenation	MUST	Semgrep rule
SEC-AP-03	Default React escaping; `dangerouslySetInnerHTML` only via DOMPurify	MUST	ESLint rule
SEC-AP-04	Strict CSP (nonces, no `unsafe-inline` scripts), `frame-ancestors 'none'`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`	MUST	Header scan in CI
SEC-AP-05	CSRF protection: SameSite cookies + CSRF token on state-changing requests	MUST	ZAP test
SEC-AP-06	SSRF prevention: worker calls allowlisted domains only; never fetches user-supplied URLs	MUST	Egress via proxy/VPC endpoints
SEC-AP-07	Uploads: magic-byte check, size limit, malware scan (ClamAV / GuardDuty Malware Protection for S3), sandboxed parsing, PDF → image to strip embedded scripts	MUST	EICAR and malicious PDF samples
SEC-AP-08	Rate limits per user/IP/endpoint; daily upload quota; daily LLM token quota per user	MUST	k6 test

Code	Requirement	Strength	Verification
SEC-AP-09	No stack traces; Problem Details with `traceId`	MUST	Test
SEC-AP-10	CORS limited to the app origin	MUST	Test
SEC-AP-11	API versioning with announced deprecation; API inventory so old endpoints stay protected	SHOULD	OpenAPI diff

13.6 AI / LLM security

Code	Requirement	Strength	Verification
SEC-AI-01	**Indirect prompt injection**: OCR text from records is *data*, wrapped in delimiters; system prompt forbids following instructions inside documents	MUST	50 images containing malicious instructions
SEC-AI-02	Redact PII (name, phone, ID number, address, patient code) before external LLM calls; re-map after	MUST	Redaction recall ≥ 99% on a labelled set
SEC-AI-03	LLM output must pass JSON Schema validation; never executed or rendered as HTML	MUST	Unit test
SEC-AI-04	Explanation retrieval only sees the user's own data (`profile_id` filter in the retrieval layer, not in the prompt)	MUST	Cross-user retrieval test
SEC-AI-05	Topic guardrails: block prescribing, dosing, survival prognosis; emergencies → 115	MUST	100-question red-team set
SEC-AI-06	LLM vendor terms (no training on data, minimal retention, processing region) recorded in a Vendor Assessment	MUST	Document

Code	Requirement	Strength	Verification
SEC-AI-07	Token limits, timeouts and circuit breakers against "denial of wallet"	MUST	Load test
SEC-AI-08	Redacted prompt/response logs, 30-day retention, restricted access	SHOULD	Langfuse config review
SEC-AI-09	Model/prompt versions stored; prompt changes go through PR + eval	MUST	CI

13.7 Infrastructure and cloud

Code	Requirement	Strength	Verification
SEC-IN-01	AWS multi-account (Organizations): management, security/log-archive, dev, staging, prod	MUST (before prod)	Account diagram
SEC-IN-02	No root usage; hardware MFA on root; human access via IAM Identity Center with temporary credentials	MUST	Security Hub
SEC-IN-03	Workloads use IAM roles (ECS task roles), least privilege, no long-lived keys; CI via GitHub OIDC → AssumeRole	MUST	IAM Access Analyzer
SEC-IN-04	RDS, Redis, workers in private subnets; only ALB public; minimal security groups	MUST	Terraform review, Config
SEC-IN-05	VPC endpoints for S3, Secrets Manager, KMS, ECR, CloudWatch so traffic stays off the internet	SHOULD	Terraform
SEC-IN-06	AWS WAF on CloudFront/ALB: managed rules (Core, Known Bad Inputs, SQLi) + rate-based rule	MUST	WAF logs

Code	Requirement	Strength	Verification
SEC-IN-07	Minimal images (distroless/slim), non-root, read-only root FS, scanned by Inspector — no deploy with Critical CVEs	MUST	CI gate
SEC-IN-08	Secrets in Secrets Manager; DB passwords rotated automatically every 30 days	MUST	Config rule
SEC-IN-09	GuardDuty, Security Hub (CIS + AWS Foundational), Config, CloudTrail (org trail into log-archive with Object Lock) on in every account	MUST	Security Hub score ≥ 90%
SEC-IN-10	IaC scanning (Checkov/tfsec) in CI; no manual console changes in prod (drift detection)	MUST	CI + Config

13.8 Logging, monitoring and incident response

Code	Requirement	Strength	Verification
SEC-LG-01	Business audit log: who, action, resource, result, IP, user-agent, traceId; append-only, kept ≥ 3 years	MUST	Test + S3 Object Lock
SEC-LG-02	Time sync (Amazon Time Sync), UTC ISO-8601 timestamps	MUST	Review
SEC-LG-03	Alerts: mass failed logins, break-glass, abnormal downloads, IAM changes, CloudTrail disabled, GuardDuty High findings	MUST	Drill
SEC-LG-04	Incident Response Plan: classify, contain, eradicate, recover, learn; notify the authority within 72 hours of detection (Decree 356/2025, Form 08) and data subjects where required	MUST	Tabletop every 6 months

Code	Requirement	Strength	Verification
SEC-LG-05	Forensic evidence preservation (snapshots, logs, chain of custody)	SHOULD	Runbook

13.9 Secure SDLC — security gates in the pipeline

Stage	Mandatory activity	Tools	Blocking condition
Requirements	Security requirements + abuse cases per epic	SRS template	Epic without abuse cases
Design	STRIDE threat model updated for new data flows	OWASP Threat Dragon	High threat without mitigation
Code	Pre-commit: gitleaks, security lint	pre-commit	Secret found
Build	SAST, SCA, IaC scan, licence scan	Semgrep, CodeQL, Trivy, Checkov	High/Critical finding
Image	Container scan, SBOM (CycloneDX), image signing	Trivy/Inspector, Syft, cosign	Critical CVE
Staging	DAST, API fuzzing	OWASP ZAP, schemathesis	High finding
Release	Security sign-off checklist	You as Security	Missing evidence
Operate	Vulnerability SLAs, yearly pentest	Inspector, Security Hub	Past SLA

13.10 Compliance mapping

Legal update: the Law on Personal Data Protection 91/2025/QH15 was passed on 26 June 2025 and took effect on 1 January 2026. **Decree 356/2025/ND-CP** guides the Law, took effect the same day and **replaced Decree 13/2023**; it sets the forms (Form 08 breach notice within 72 hours of detection, Form 09 overseas-transfer impact assessment, Form 10 processing impact assessment) and the qualifications of the data protection officer. Consents, DPIAs and dossiers made before 1 January 2026 should be reviewed against the new rules. Key points: consent must be voluntary, informed and specific to each purpose; the DPIA and the overseas-transfer impact assessment (OTIA) are filed within

60 days of starting processing/transfer. This is a learning summary, not legal advice. Sources: Law 91/2025/QH15, Allen & Gledhill, Luat Viet An — what is new in Decree 356/2025.

Requirement / standard	Main content	Met by
PDPL – Consent	Explicit, purpose-specific consent for sensitive data; withdrawable	BR-002, BR-034, `consents` table
PDPL – Data subject rights	Access, correction, deletion, restriction, objection, consent withdrawal	Export, edit, account deletion, BR-032
PDPL – DPIA	Processing impact assessment filed within 60 days	DPIA (section 6.9)
PDPL – Breach	Notify within the legal deadline	SEC-LG-04
PDPL – Overseas transfer	OTIA filed within 60 days of starting transfer	Vendor Assessment, nearby region, PII redaction
HIPAA Security Rule (reference)	Administrative / physical / technical safeguards	13.2–13.8, HIPAA-eligible AWS services
OWASP ASVS L2	~280 verification requirements	Checklist in `docs/security/asvs.md`
OWASP Top 10 for LLM	Prompt injection, data leakage, excessive agency…	13.6

Note: this is a learning document, not legal advice. Before putting other people's real data into the system, consult a lawyer or data-protection specialist.

14. Data requirements

All MediLens data must be **classified, owned, retention-bound, quality-measured and traceable to its source**. This applies to BE, DE and AI alike; DE enforces and reports.

14.1 Data classification

Class	Examples	Stored in	Mandatory controls
C4 – Sensitive health (PHI)	Record images, OCR text, diagnoses, lab values, BodyFindings	RDS (prod), S3 `phi-*` buckets	Dedicated KMS CMK, RLS, every read audited, never leaves prod, never in logs
C3 – Personal identifiers (PII)	Name, email, phone, DOB, IP	RDS (field-encrypted), Cognito/Keycloak	Field encryption, masked in logs, deleted on request
C2 – Internal	Unit configs, prompts, AI metrics, redacted logs	RDS, S3 `internal-*`, CloudWatch	IAM by role
C1 – Public	Approved medical articles, 3D/2D assets, marketing pages	S3 + CloudFront	Integrity (only CI writes)
C0 – Synthetic	Fake records, golden set	S3 `ml-*`, repo (no real images)	Tagged `synthetic=true`, never mixed with C4

14.2 Data requirements (DAT-xxx)

Code	Requirement	Strength
DAT-01	Every table, bucket and topic is in the Data Catalog with owner role, class, description and retention	MUST
DAT-02	Every clinical record traces to `document_id` + `extraction_run_id` + page + bounding box (record-level lineage)	MUST
DAT-03	AI value (`ai_value`) and user-confirmed value (`user_value`) stored separately; never overwritten	MUST
DAT-04	All timestamps UTC (`timestamptz`), shown in the user's time zone; visit dates as timezone-less `date`	MUST

Code	Requirement	Strength
DAT-05	Units normalised to UCUM (mmol/L, mg/dL…), keeping original value + unit	MUST
DAT-06	Standard codes: ICD-10 (diagnoses), LOINC (labs), SNOMED CT/FMA (anatomy), FDI (teeth), TNM (cancer); code tables versioned with effective dates	MUST
DAT-07	Catalogues (Department/Unit/Region/standard codes) are **master data**: changed only via PR + migration, with history	MUST
DAT-08	Analytics data pseudonymised with HMAC using a separate key; no C3	MUST
DAT-09	Statistics published only for groups ⩾ 10 people (k-anonymity k ⩾ 10)	MUST
DAT-10	User data for evaluation/training only with opt-in consent, de-identified, documented in a Dataset Card	MUST
DAT-11	Schema changes follow expand → migrate → contract; never drop a column in the release that adds its replacement	MUST
DAT-12	Backups proven restorable on a schedule (see OPS)	MUST
DAT-13	User data export in machine-readable form (JSON; FHIR Bundle in v2)	SHOULD
DAT-14	Data contracts between producer and consumer (schema, semantics, SLA, quality) for every feed into analytics	SHOULD

14.3 Data lifecycle and retention

Data	Created	Used for	Hot storage	Archive	Deletion
Original images/PDFs	Upload	OCR, review, re-viewing	S3 Standard until user deletes	S3 Intelligent-Tiering moves cold objects automatically	On account deletion: soft at 0 days, hard at 30 days
OCR text, extraction	Pipeline	Review, evaluation	RDS	—	With the source document
Confirmed clinical data	User confirm	Viewer, timeline	RDS	—	With the account
Incomplete uploads	Presign	—	24 hours	—	S3 lifecycle rule
Audit log	Every access	Investigation, compliance	CloudWatch 90 days	S3 + Object Lock 3 years	Expires
Application logs	Runtime	Debugging	30 days	1 year (redacted)	Expires
LLM prompts/responses (redacted)	Pipeline	Debug, eval	30 days	—	Expires
Analytics	ETL	Dashboards	2 years	—	Rolling
DB backups	AWS Backup	DR	35-day PITR	Monthly copy for 1 year in a separate vault account	Expires

14.4 Data quality (6 dimensions)

Dimension	Definition	Example check	Threshold
Completeness	Required fields present	`condition.icd10_code` not null when confirmed	100%
Validity	Correct format/domain	Tooth ∈ {11–18, 21–28, 31–38, 41–48, 51–85}; ICD-10 exists in code table	100%
Accuracy	Matches reality	Golden set; user correction rate	See section 10
Consistency	No contradictions	Sex vs organ; `observed_at` ≥ birth date	100%

Dimension	Definition	Example check	Threshold
Timeliness	Fresh enough	Analytics updated < 24 h; documents processed < 2 min	SLA
Uniqueness	No duplicates	Document `sha256`; one Condition per Encounter + code	100%

Checks run in three places: **on write** (BE validation), **in pipelines** (dbt tests / Great Expectations), **periodically** (data-quality report cron). MUST violations stop the pipeline and alert; SHOULD violations go into the report.

14.5 Data for AI

- **Golden set:** ⩾ 200 synthetic + public documents stratified by Unit × document type × image quality (good / medium / poor) × printed/handwritten; version-locked, never used for tuning.
- **Dev set:** ⩾ 300 documents for development; **red-team set:** 100 questions + 50 prompt-injection images.
- **Annotation guideline:** field definitions, right/wrong examples, rules for ambiguous cases; measure agreement (Cohen's kappa ⩾ 0.8 when self-relabelling).
- **Dataset Card** per set: source, licence, creation process, distribution, limitations, date, version (S3 + DVC or S3 versioning).
- **Never** put real C4 data into the golden set without consent and de-identification.

14.6 Data responsibilities (condensed RACI)

Activity	DE	BE	AI	Security	PO
OLTP schema	C	R/A	C	C	I
Data catalog & classification	R/A	C	C	C	I
Analytics pipelines	R/A	I	C	C	I
AI datasets	C	I	R/A	C	I
Retention & deletion policy	R	R	I	A	C
Data subject requests	C	R	I	A	I

15. Data Engineering in detail

MediLens DE serves four needs: **(1) standard catalogues** for the app, **(2) AI data** (synthetic, golden sets, embeddings), **(3) product and AI-quality analytics**, **(4) compliance** (lineage, retention, deletion). Analytics follows the **medallion model (bronze → silver → gold)** on S3, fully separated from the operational database.

15.1 Data layers

Layer	Content	Format	Location	Readers
Source (OLTP)	Business tables	PostgreSQL	RDS prod	App only
Bronze (raw)	Change copies (CDC) or daily exports **pseudonymised at extraction**, app events	Parquet, `dt=` partitions	S3 `analytics-bronze`	DE
Silver (clean)	Cleaned, typed, deduplicated, joined with catalogues, contract-checked	Parquet / Apache Iceberg	S3 `analytics-silver`	DE, AI
Gold (mart)	Fact/dim tables for dashboards and AI eval	Iceberg / Postgres analytics	S3 `analytics-gold` + Athena	PO, AI, dashboards
ML datasets	Golden/dev/red-team, synthetic, labels	JSONL + images, versioned	S3 `ml-datasets`	AI
Vector	Chunks + embeddings of approved articles	pgvector	RDS	API (RAG)

15.2 Pipelines

Pipeline	Source → Target	Schedule	Tooling	SLA
`ref_catalog_load`	ICD-10, LOINC subset, abbreviation dictionary, Unit mapping → master tables	On PR change	Python + Alembic data migration	Before deploy
`synthetic_generate`	Unit templates + LLM → fake records (PDF/images) + labels	On demand	Prefect flow	—
`dataset_build`	Synthetic + public + Label Studio exports → versioned golden/dev sets	On new labels	Prefect + DVC/S3 versioning	—
`oltp_extract`	RDS → Bronze (HMAC pseudonymisation, C3 dropped)	02:30 daily	AWS Glue job or Python on ECS; DMS CDC later	Done before 03:00
`events_ingest`	Product events (outbox) → Bronze	Continuous / 15 min	Kinesis Data Firehose → S3	⩽ 15 min lag
`transform_silver_gold`	Bronze → Silver → Gold	03:00 daily	dbt (dbt-athena or dbt-postgres)	Done before 06:00
`ai_quality_mart`	`extraction_runs` + `extraction_fields` + corrections → AI facts	03:30 daily	dbt	Before 06:00
`knowledge_index`	Approved articles → chunks → embeddings → pgvector	On approval	Worker job	⩽ 10 min
`retention_enforce`	Delete/archive per table 14.3	Monthly	Python + S3 Lifecycle	Report

Pipeline	Source → Target	Schedule	Tooling	SLA
`dsr_delete`	Data subject deletion across OLTP, S3, analytics, vectors	On event	Worker + Prefect	⩽ 30 days

15.3 Gold model (star schema)

- **fact_document_processing**: document_key, unit_key, date_key, doc_type, pages, status_final, time_to_extract_s, time_to_confirm_s, ocr_engine, model_version_key, cost_usd.
- **fact_field_extraction**: run_key, field_path, unit_key, confidence_bucket, was_corrected, correction_type (wrong value / missing / extra), match_exact, match_fuzzy.
- **fact_user_activity**: user_pseudo_id, date_key, event_type, session_id, device_class.
- **fact_explain_query**: date_key, unit_key, guardrail_triggered, citations_count, latency_ms, tokens, feedback.
- **dim_unit, dim_date, dim_model_version, dim_prompt_version, dim_doc_type, dim_region.**

Every metric in section 10 must be computable from Gold and defined once in a **metrics layer** (dbt semantic layer or a YAML metric file) so all dashboards share one formula.

15.4 DE requirements (DE-xxx)

Code	Requirement	Strength
DE-01	Idempotent pipelines: re-running a day gives the same result (overwrite by partition)	MUST
DE-02	Every pipeline has an owner, schedule, SLA, failure alert and dead man's switch	MUST
DE-03	Data leaving RDS is pseudonymised before Bronze; the HMAC key lives in Secrets Manager, readable only by DE jobs	MUST
DE-04	Bronze/Silver schemas managed (Glue Data Catalog); breaking changes fail the pipeline	MUST

Code	Requirement	Strength
DE-05	dbt tests on every Silver/Gold model: unique, not_null, accepted_values, relationships + custom tests from 14.4	MUST
DE-06	Documented, date-ranged, duplicate-free backfills	MUST
DE-07	Column lineage from source to dashboard (dbt docs / OpenLineage)	SHOULD
DE-08	Athena cost capped with workgroups (bytes-scanned limit)	SHOULD
DE-09	ML datasets are immutable versions with a Dataset Card and checksums	MUST
DE-10	Weekly data-quality report to PO + AI	SHOULD

15.5 Example data contract (`ExtractionConfirmed` event)

```
contract: extraction_confirmed.v1
owner: backend
consumers: [analytics, ai-eval]
classification: C2   # pseudonymised, no raw clinical values
schema:
  event_id:        {type: uuid, required: true}
  occurred_at:     {type: timestamp_utc, required: true}
  user_pseudo_id:  {type: string, required: true, note: HMAC-SHA256}
  document_id:     {type: uuid, required: true}
  unit_code:       {type: string, required: true, enum_ref: units}
  fields_total:    {type: int, min: 1}
  fields_corrected:{type: int, min: 0}
  model_version:   {type: string, required: true}
  prompt_version:  {type: string, required: true}
quality:
  - fields_corrected <= fields_total
  - freshness: delivered within 15 minutes
sla: 99% events delivered within 15 minutes
breaking_change_policy: new version v2, run v1 and v2 in parallel for 30 days
```

15.6 DE tools and services by phase

Phase	Budget option	Enterprise option (learning)
MVP	Separate Postgres `analytics` schema + dbt-postgres + cron + Metabase	—
v1.5	S3 + Parquet + Athena + dbt-athena + Prefect on ECS	Glue Data Catalog, Lake Formation column permissions
v2	Iceberg tables, Firehose for events, QuickSight/Superset	DMS CDC, MWAA (managed Airflow), OpenLineage

15.7 DE-specific risks

- PHI leaking into analytics → pseudonymise at source, Lake Formation column blocks, data-contract review.
- Pipelines "green" but data wrong → semantic tests (correction rate cannot be negative, totals match OLTP ±0.5%).
- Athena cost growth from full scans → date partitions, Parquet, workgroup limits.
- Out-of-sync deletion across stores → `dsr_delete` pipeline with a per-store checklist and completion report.

16. Solution Architecture

The Solution Architect owns **every decision that is hard to reverse**: system boundaries, integration patterns, platform choices and quality attributes. The main outputs are a **Solution Architecture Document (SAD)** using C4 + arc42, the ADR set and measurable NFRs.

16.1 Non-functional requirements (NFR)

Code	Attribute	Measurable requirement	How achieved	Verification
NFR-01	Availability	API 99.5%/month (≤ 3h36m downtime)	Multi-AZ ALB/ECS, RDS Multi-AZ in prod	CloudWatch SLO

Code	Attribute	Measurable requirement	How achieved	Verification
NFR-02	Latency	Read API p95 ⩽ 300 ms, p99 ⩽ 1 s	Indexes, Redis cache, pagination	k6
NFR-03	Throughput	50 req/s sustained, 200 req/s 5-min peak	ECS autoscaling on CPU + request count	Load test
NFR-04	Document processing	95% done < 2 min; 100 documents/hour	SQS + worker autoscaling on queue depth	Pipeline load test
NFR-05	Scalability	10× users without re-architecture	Stateless API, separate worker, S3 storage	Review
NFR-06	Durability	0 loss of successfully uploaded documents	S3 (11 nines), acknowledge only after DB write	Chaos test
NFR-07	Recoverability	RPO ⩽ 24 h (target 5 min with PITR), RTO ⩽ 4 h	AWS Backup, PITR, IaC rebuild	DR drill
NFR-08	Security	ASVS L2, 0 High open past SLA	Section 13	Security sign-off
NFR-09	Privacy	PDPL 91/2025 + Decree 356/2025 compliant	Sections 13–14	DPIA
NFR-10	Accessibility	WCAG 2.2 AA	Design System, axe	Audit
NFR-11	Usability	SUS ⩾ 70; newcomers finish the main flow ⩽ 5 min	Usability tests	Report
NFR-12	Maintainability	New Unit ⩽ 3 person-days, no core changes	Unit config + viewer plugins	Trial in v1.5
NFR-13	Observability	100% requests carry traceId; every alert has a runbook	OpenTelemetry	Review

Code	Attribute	Measurable requirement	How achieved	Verification
NFR-14	Portability	Switch LLM/OCR provider ⩽ 2 days	Adapter pattern, eval harness	Provider swap trial
NFR-15	Cost	⩽ 100 USD/month at 100 MAU; AI ⩽ 0.10 USD/document	Fargate Spot for workers, caching	Cost report
NFR-16	i18n	Full VI + EN, easy to add languages	next-intl, bilingual medical content	Review
NFR-17	Compatibility	Last 2 versions of Chrome, Safari, Firefox, Edge; iOS 16+, Android 10+	Test matrix	Playwright

16.2 Quality attribute scenarios (examples)

- **Availability:** *When* one AZ loses connectivity *at peak*, *the system* keeps serving the API with < 1% 5xx and self-heals within 5 minutes, losing no in-flight documents (messages return to the queue).
- **Security:** *When* user A changes the URL ID to B's document, *the system* returns 404, writes an audit event and alerts at ⩾ 10 attempts/minute.
- **Modifiability:** *When* adding the Nephrology–Urology Unit, *the team* adds only config, 2D/3D assets, an extraction schema and content, *in* ⩽ 3 person-days, without redeploying the core API.
- **Cost:** *When* traffic grows 5× in a day, *AI spend* stays within the daily budget thanks to per-user quotas and circuit breakers.

16.3 ADR backlog

#	Decision	Options	Recommendation
ADR-001	Architecture style	Monolith / Modular monolith / Microservices	Modular monolith + AI worker
ADR-002	Backend language	Python FastAPI / NestJS / Spring Boot	FastAPI

#	Decision	Options	Recommendation
ADR-003	Identity provider	Self-hosted Keycloak / Amazon Cognito / Auth0	Cognito on AWS (less ops); Keycloak to learn OIDC deeply
ADR-004	Compute	ECS Fargate / EKS / Lambda / App Runner	ECS Fargate (API + worker)
ADR-005	Queueing	SQS / Redis (Celery) / EventBridge	SQS for jobs, EventBridge for domain events
ADR-006	OCR	Self-hosted PaddleOCR / Textract / Document AI / vision LLM	Hybrid: PaddleOCR/VietOCR primary; Bedrock vision LLM for handwriting; Textract only for English documents (Textract does not support Vietnamese)
ADR-007	LLM	Amazon Bedrock / direct API / self-hosted	Bedrock (data stays in AWS, IAM, VPC endpoint)
ADR-008	Vector store	pgvector / OpenSearch / Bedrock Knowledge Bases	pgvector
ADR-009	3D format	glTF/GLB + Draco / FBX / USDZ	glTF/GLB
ADR-010	Event integration	Transactional outbox / dual-write	Outbox
ADR-011	Tenancy	None (B2C) / per organisation	Single-tenant B2C, `org_id` reserved for B2B
ADR-012	AWS region	ap-southeast-1 Singapore / ap-southeast-7 Thailand / other	Singapore (full service set, close to VN); re-check Bedrock model availability
ADR-013	Health data standard	Custom / FHIR-aligned	FHIR-aligned naming, FHIR export in v2
ADR-014	Frontend rendering	SSR / SPA / hybrid	Next.js hybrid; viewer client-only

ADR format (Michael Nygard): *Title, Status, Context, Decision, Consequences, Alternatives considered, Date*.

16.4 Integration and resilience patterns

- **Async request–reply:** upload returns 202 + `document_id`; the FE follows progress via SSE; no HTTP request is held open during OCR.
- **Retry + idempotency:** every consumer handles duplicates safely (idempotency key = `event_id`); exponential backoff + jitter; after N tries → DLQ + alert.
- **Circuit breaker + fallback:** LLM provider errors > 50% in 1 minute → open circuit, switch to a backup provider or hold jobs in the queue.
- **Bulkhead:** OCR and LLM workers are separate services/queues so one failing doesn't block the other.
- **Simple saga:** account deletion = compensable steps (mark → delete S3 → delete DB → delete analytics → delete vectors → confirm).
- **Strangler/Adapter:** every external service (OCR, LLM, email) behind a `Provider` interface so it can be replaced.

16.5 Environments and release

Environment	Purpose	Data	Scale	Deploy
local	Development	Synthetic seed	Docker Compose	Manual
dev	Continuous integration	Synthetic	Smallest, off at night	Auto on every merge
staging	Pre-release testing, UAT, load tests	Large synthetic set	Like prod (single-AZ allowed)	Auto, then release tag
prod	Real users	Real	Multi-AZ	Manual approval, blue/green

Feature flags (AWS AppConfig or Unleash) toggle Units/features without deploys; a 2-week release train per sprint; hotfixes use a separate branch but still pass CI.

16.6 Architecture governance (even solo)

- **Architecture Review** at the end of every phase: check NFRs, ADRs, risks; keep minutes.

- **Automated fitness functions:** module-boundary tests (import-linter), FE bundle size, staging p95 latency, cost per day.
- **Personal tech radar:** Adopt / Trial / Assess / Hold for project technologies.
- **Definition of Architecture Done:** C4 diagrams updated, ADRs complete, NFRs tested, threat model updated, runbooks exist.

17. Backend in detail

The backend must be **correct (authorization, data), durable (jobs, retries), observable (logs/traces/metrics) and extensible (new Units without core changes)**. This section is the "coding standard + API standard" an enterprise backend team would sign off.

17.1 Module structure (Clean / Hexagonal inside a modular monolith)

```
apps/api/src/medilens/
├── core/                # config, logging, security, db session, errors,
telemetry
├── shared/              # shared types, pagination, idempotency, outbox
├── modules/
│   ├── identity/
│   │   ├── api/         # FastAPI router, DTOs (Pydantic)
│   │   ├── application/  # use cases / services, authorization policies
│   │   ├── domain/      # entities, value objects, domain events, rules
(BR-xxx)
│   │   └── infra/       # SQLAlchemy repositories, external adapters
│   ├── patient/  document/  extraction/  anatomy/  clinical/
│   ├── knowledge/  sharing/  notification/  audit/  admin/
└── main.py
apps/worker/src/medilens_worker/
├── jobs/                # validate, preprocess, classify, ocr, extract,
materialize…
├── providers/           # ocr_paddle.py, ocr_textract.py, llm_bedrock.py
(Provider interface)
└── pipeline/            # state machine, retries, DLQ
```

Dependency rule: `api → application → domain`; `infra` implements `domain` interfaces. Modules talk only through public `application` services or domain events — **never** querying another module's tables. Enforced with import-linter in CI.

17.2 API standards (BE-API-xxx)

Code	Standard	Strength
BE-API-01	OpenAPI 3.1 is the source of truth; FE client generated; API-changing PRs include an OpenAPI diff	MUST
BE-API-02	Plural kebab-case nouns: `/api/v1/medical-documents/{id}/pages`	MUST
BE-API-03	URL versioning (`/v1`); breaking change → `/v2` , keep v1 ≥ 90 days with `Deprecation` and `Sunset` headers	MUST
BE-API-04	JSON fields `camelCase` in the API, `snake_case` in the DB; ISO-8601 UTC times	MUST
BE-API-05	Cursor pagination (`?limit=&cursor=`), max limit 100	MUST
BE-API-06	Allowlisted filtering/sorting (`?status=confirmed&sort=-observedAt`)	MUST
BE-API-07	Errors as RFC 9457 Problem Details: `type` , `title` , `status` , `detail` , `instance` , `traceId` , `errors[]`	MUST
BE-API-08	`Idempotency-Key` required on important resource-creating POSTs (upload complete, share, export)	MUST
BE-API-09	Optimistic concurrency: `ETag` + `If-Match` on PATCH of extraction/condition	MUST
BE-API-10	Long tasks return `202 Accepted` + `Location` to a status resource	MUST
BE-API-11	Rate-limit headers `RateLimit-Limit` , `RateLimit-Remaining` , `Retry-After`	SHOULD
BE-API-12	Stream LLM explanations via SSE with heartbeat and cancellation	SHOULD

Business error codes (excerpt):

HTTP	`type`	When
400	`/errors/validation`	Schema mismatch
401	`/errors/unauthenticated`	Missing/expired token
403	`/errors/consent-required`	Terms not accepted (BR-002)
404	`/errors/not-found`	Does not exist **or no permission** (avoid revealing existence)
409	`/errors/duplicate-document`	Duplicate document (BR-016)
409	`/errors/version-conflict`	ETag mismatch
413	`/errors/file-too-large`	> 20 MB (BR-010)
415	`/errors/unsupported-media`	Not JPG/PNG/HEIC/PDF
422	`/errors/low-quality-image`	Blurry image (BR-011)
422	`/errors/unconfirmed-fields`	Yellow fields still unconfirmed (BR-012)
429	`/errors/rate-limited`	Quota exceeded
503	`/errors/ai-unavailable`	AI circuit breaker open

17.3 Coding and quality standards (BE-CODE-xxx)

- Python 3.12, full type hints, `mypy --strict` for `domain` and `application`; `ruff` (lint + format); no `print`, use the structured logger.
- Async end-to-end (asyncpg, httpx); no blocking I/O in the event loop; CPU-bound work (OCR, image processing) only in the worker.
- Transactions at the application layer, one use case = one transaction; outbox written in the same transaction.
- Every BR-xxx has a clearly named function/rule (`ensure_fields_confirmed()`) and a unit test named with the rule code.
- Configuration via environment variables + Pydantic Settings; secrets read from Secrets Manager at startup; no default values for secrets.

- JSON logging: `timestamp, level, service, env, traceId, spanId, userPseudoId, route, status, latencyMs` — never PHI.
- Alembic migrations: every migration has a downgrade (or a stated reason why not) and is rehearsed on a staging copy in CI.

17.4 Performance and data access

Area	Standard
Queries	No N+1 (use `selectinload`), every `profile_id` query indexed, `EXPLAIN` for queries > 50 ms
Connection pool	Sized per ECS task count; RDS Proxy when connections grow
Cache	Redis/ElastiCache for unit configs (TTL 10 min, invalidated on publish) and rate-limit counters; **never cache PHI**
Payload	Gzip/Brotli; lists return summaries, details via separate endpoints
Timeouts	Outbound calls: connect 2 s, read 30 s (LLM 120 s), bounded retries
Budget	API p95 ⩽ 300 ms; timeline endpoint ⩽ 500 ms with 5 years of data

17.5 Backend testing

Level	Scope	Tools	Threshold
Unit	Domain, BR rules, mappers	pytest, hypothesis (property-based for tooth/unit parsers)	Domain coverage ⩾ 90%
Integration	Repositories, outbox, RLS, S3 presign	pytest + Testcontainers (Postgres, LocalStack)	Main flows 100%
Contract	OpenAPI ↔ implementation, FE ↔ BE	schemathesis, Pact (optional)	0 violations

Level	Scope	Tools	Threshold
Authorization	Role × endpoint × ownership matrix	pytest parametrize generated from OpenAPI	100% endpoints
Worker	Each job + full chain with fake providers	pytest, moto	Error paths + retries + DLQ
Performance	Read/write APIs, upload	k6	NFR-02/03
Migration	Upgrade/downgrade on a populated DB	Alembic + Testcontainers	Pass

17.6 Backend PR review checklist

- [ ] New endpoint has an authorization policy + IDOR test
- [ ] Input validated by DTO; output doesn't expose internal fields
- [ ] No PHI/PII logged; errors return Problem Details
- [ ] Transaction + outbox correct; consumers idempotent
- [ ] Migration backward compatible
- [ ] OpenAPI updated, FE client regenerated
- [ ] Metrics/traces for the new path
- [ ] Unit + integration tests; coverage not lower

18. Frontend in detail

The frontend is where patients *understand* their condition, so the FE standard rests on four pillars: **clear and not frightening, fast, accessible, secure**. The 3D/2D viewer is the most complex component and has its own asset pipeline.

18.1 Application architecture

```
apps/web/src/
├── app/[locale]/                 # Next.js App Router, VI/EN
|   ├── (public)/                 # landing, about, privacy — SSR/SSG
|   ├── (auth)/                   # login, register, mfa
|   └── (app)/                    # dashboard, documents, review, viewer,
timeline, share, settings, admin
├── features/                     # by domain: documents/, review/, viewer/,
timeline/, explain/, sharing/
|   └── viewer/
|       ├── core/                 # Canvas, camera, controls, raycast,
highlight, legend
|       ├── units/                # plugins: dental/, cardiology/, oncology/
(Viewer2D, Viewer3D, DetailPanel)
|       └── registry.ts           # unit plugin registration
├── components/ui/                # Design System (shadcn) — no business
logic
├── lib/api/                      # OpenAPI-generated client + TanStack Query
hooks
├── lib/auth/                     # OIDC (Auth.js or Amplify), guards
└── styles/tokens.css             # design tokens (BR-020 severity colours,
spacing, typography)
```

- **Server state:** TanStack Query (caching, retries, invalidation by keys like `['documents', id]`); **UI state:** Zustand (camera preset, selected Unit, x-ray mode); **forms:** React Hook Form + Zod (schemas shared with the API client).
- **Rendering:** public pages SSG; app pages render client-side after authentication; viewer via `dynamic(() => import(...), { ssr: false })`.
- **Realtime:** EventSource (SSE) for document status with auto-reconnect and a 5-second polling fallback.

18.2 3D/2D asset pipeline (the heart of FE work)

1. **Sources:** Z-Anatomy/BodyParts3D (CC BY-SA), own Blender models; licences recorded in `ATTRIBUTION.md`.
2. **Normalise in Blender:** metres, Y-up, origin at the pelvis centre; split meshes by the Unit config's `region_code` (e.g. `tooth_36`, `heart_lv`, `lung_r_upper`); mesh name = `region_code`.

3. **Optimise:** decimate to polygon budgets (full body ⩽ 150k triangles, detailed Unit ⩽ 80k), merge materials, textures ⩽ 1024 px KTX2.
4. **Export:** binary glTF 2.0 (`.glb`), Draco or Meshopt compression via `gltf-transform`; two LOD levels.
5. **Automated CI checks:** a script reads the GLB, compares mesh names with the Unit config `regions[]` → missing/extra fails; size ⩽ 5 MB; render thumbnails for visual regression.
6. **2D SVG:** each region is a `<path id="region_code">` with bilingual `<title>` for screen readers; optimise with SVGO keeping `id`s.
7. **Delivery:** S3 + CloudFront, immutable cache keyed by file hash, correct `Content-Encoding`.

18.3 Viewer requirements (FE-VW-xxx)

Code	Requirement	Strength
FE-VW-01	Rotate/zoom/pan with mouse, touch (pinch) and keyboard (arrows, +/−); reset-camera button	MUST
FE-VW-02	Click/tap a region → detail panel; hover → VI/EN region name tooltip	MUST
FE-VW-03	Severity colours (BR-020) with icon/label; legend always visible	MUST
FE-VW-04	X-ray mode (skin/muscle transparency) to reveal organs; layer toggles: bones, muscles, organs, vessels, lymph nodes	MUST
FE-VW-05	Metastasis: lines from primary organ → metastatic site (BR-021), separate "reference" layer	MUST (Oncology)
FE-VW-06	Full text alternative listing all regions (a11y)	MUST
FE-VW-07	Timeline: date slider, side-by-side compare of two dates, state-change animation	SHOULD
FE-VW-08	Weak/no WebGL detected → switch to 2D automatically with a gentle notice	MUST
FE-VW-09	Capture viewer image to share with a doctor (with watermark)	MAY

Code	Requirement	Strength
FE-VW-10	`prefers-reduced-motion` → animations off	MUST

18.4 Performance budgets

Metric	Budget
Initial JS (app route)	⩽ 200 KB gzip; viewer chunk loaded separately ⩽ 350 KB gzip
LCP / INP / CLS	⩽ 2.5 s / ⩽ 200 ms / ⩽ 0.1 (real-user p75 via web-vitals → CloudWatch RUM)
3D model	⩽ 5 MB, loads ⩽ 3 s on 4G; 2D shown while loading
FPS	⩾ 50 desktop, ⩾ 30 mid-range mobile; ⩽ 100 draw calls
GPU memory	Dispose geometries/textures on leaving the page (avoid leaks)

CI blocks PRs that exceed bundle budgets (size-limit) or score < 90 in Lighthouse CI on public pages.

18.5 Accessibility and content

- WCAG 2.2 AA: contrast ⩾ 4.5:1, visible focus, keyboard navigation for every flow, form labels, errors tied to fields, `aria-live` for processing status.
- Language at grade 6–8 (plain language); medical terms get explanatory tooltips; always the disclaimer "illustration, not a substitute for your doctor".
- Sensitive content (cancer, metastasis): open with a calm summary; details behind "Show details"; offer support links.
- i18n: no string concatenation; dates/numbers per locale; RTL not needed in v1.

18.6 Frontend security (FE-SEC-xxx)

Code	Requirement	Strength
FE-SEC-01	Tokens only in HttpOnly cookies managed by the BFF/route handlers; never localStorage	MUST
FE-SEC-02	CSP with nonces, no inline scripts; Trusted Types where possible	MUST
FE-SEC-03	Never render HTML from the LLM; markdown through a safe renderer + DOMPurify	MUST
FE-SEC-04	No PHI in URL queries (use IDs), analytics or RUM	MUST
FE-SEC-05	Dependency audit, lockfile, scripts only from the app's own domain	MUST
FE-SEC-06	Clear TanStack Query cache and state on logout	MUST

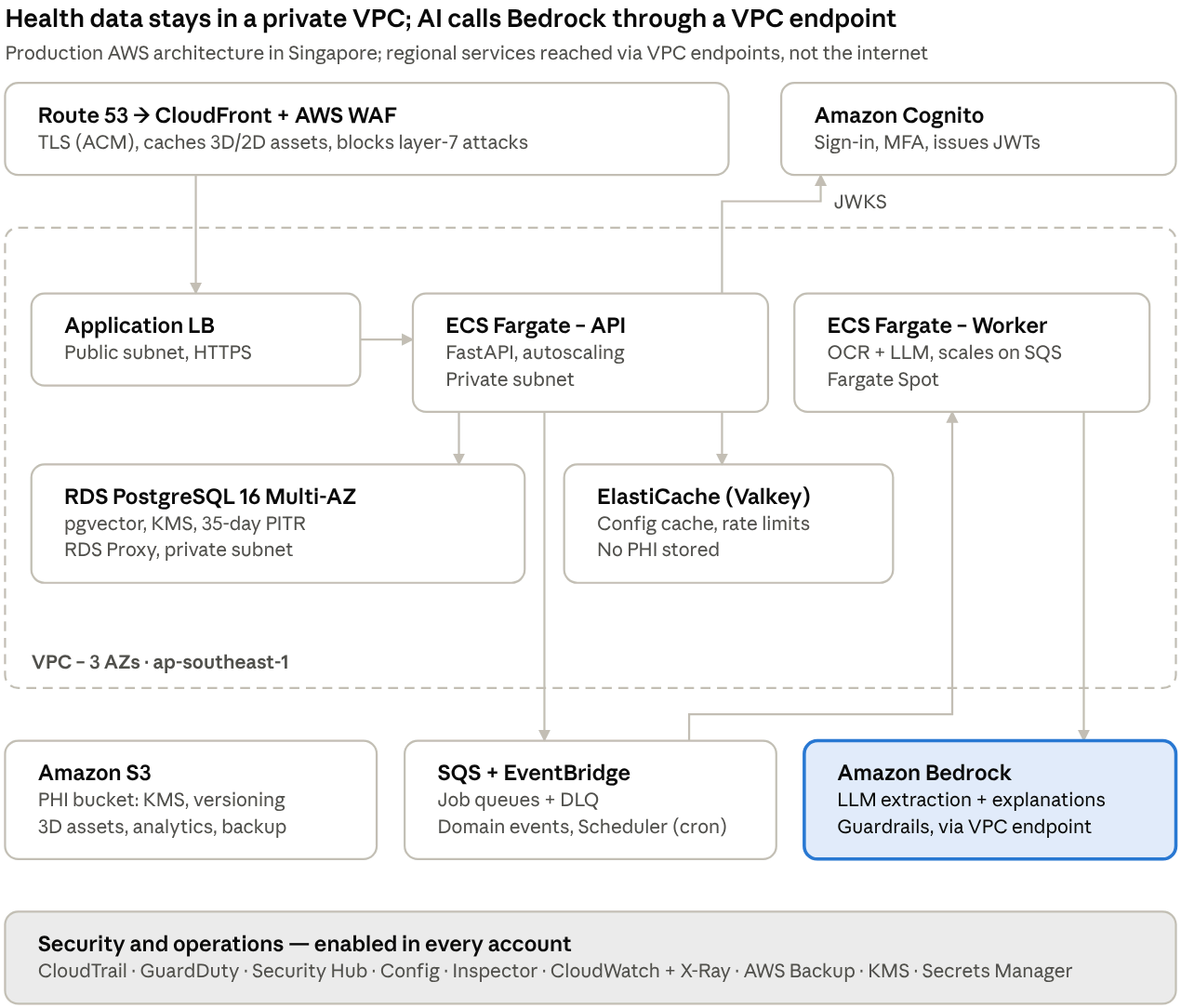
18.7 Frontend testing

Level	Tool	Scope
Unit/component	Vitest + Testing Library	Hooks, forms, Design System components, severity colour mapping
Storybook + interaction	Storybook test runner, Chromatic (optional)	Every UI component, loading/empty/error states
Viewer	Playwright screenshots at fixed camera presets, `toHaveScreenshot` with threshold	Each Unit, each severity
E2E	Playwright with mocked API (MSW) and against real staging	Sign-up → upload → review → viewer → share
A11y	axe-core in Playwright, manual NVDA/VoiceOver check each release	0 serious issues

Level	Tool	Scope
Cross-browser	Playwright projects: Chromium, WebKit, Firefox, mobile viewports	Main flows

19. Choosing AWS services

Recommendation: **ECS Fargate + RDS PostgreSQL + S3 + SQS + Bedrock + Cognito** in **Singapore (ap-southeast-1)**, built with Terraform across multiple accounts via AWS Organizations. This set is common in enterprises, light to operate for one person, and keeps health data inside AWS (even when calling an LLM). Bedrock is available in both Singapore and Thailand (ap-southeast-7), but model lists differ per region — check the Regional availability page before finalising ADR-012.



production AWS architecture · 3-AZ VPC, services via VPC endpoints

The worker reads files from S3 and writes results to RDS (not drawn, to keep the diagram readable); the API only issues presigned URLs, so files go straight from the browser to S3.

19.1 Account structure (landing zone)

Account	Purpose	Notes
management	AWS Organizations, billing, IAM Identity Center	No workloads
security	GuardDuty/Security Hub delegated admin, IAM Access Analyzer	Read-only

Account	Purpose	Notes
log-archive	CloudTrail org trail, Config, VPC Flow Logs, application audit logs	S3 Object Lock
shared-services	ECR, Route 53 hosted zone, artefacts	Optional
dev / staging	Development, testing	Synthetic data only; off out of hours
prod	Real users	Multi-AZ, strict SCPs
backup-vault	Cross-account AWS Backup vault	Protects against ransomware/accidental deletion

Minimum SCPs: deny unused regions, deny disabling CloudTrail/GuardDuty/Config, deny creating IAM users with access keys, deny sharing snapshots outside the org, require S3/EBS encryption. Early on, 3 accounts (management, nonprod, prod) are enough, then split; AWS Control Tower speeds this up if you want to learn it.

19.2 Network design

- One `/16` VPC per environment (dev 10.10.0.0/16, staging 10.20.0.0/16, prod 10.30.0.0/16), 3 AZs.
- Three subnet tiers per AZ: **public** (ALB, NAT), **private-app** (ECS tasks), **private-data** (RDS, ElastiCache — no internet route).
- NAT Gateway: one in dev/staging (cheaper), one per AZ in prod; package updates go via NAT, AWS services via **VPC endpoints** (Gateway for S3; Interface for SQS, Bedrock Runtime, Secrets Manager, KMS, ECR, CloudWatch Logs, STS).
- Security groups reference each other: ALB → API (8000), API/Worker → RDS (5432), API → ElastiCache (6379); nothing open to 0.0.0.0/0 except ALB 443.
- VPC Flow Logs → log-archive; Route 53 Resolver DNS Firewall (optional) blocks malicious domains.

19.3 Service selection table

Need	Service	Why	Alternatives (when)
DNS, CDN, attack protection	Route 53, CloudFront, AWS WAF, ACM, Shield Standard	Free TLS, 3D assets cached near users, WAF managed rules	Cloudflare (cheaper, outside AWS)
Next.js frontend	ECS Fargate (`web` container) behind ALB; static assets via S3 + CloudFront	One deploy model for every service	AWS Amplify Hosting (fastest), Vercel (outside AWS)
API	ECS Fargate + ALB	No servers to manage, autoscaling, learn containers	App Runner (simpler), Lambda + API Gateway (low traffic), EKS (to learn K8s)
AI worker	ECS Fargate (Spot for retryable work), scaling on SQS depth	OCR needs RAM/CPU and runs longer than Lambda limits	AWS Batch / ECS on GPU EC2 (self-hosted CV/LLM), SageMaker endpoints
Authentication	Amazon Cognito (User Pool, MFA, Hosted UI/OIDC)	Managed, IAM integration, priced per MAU	Keycloak on ECS (learn OIDC deeply, self-operated)
Database	RDS for PostgreSQL 16 (Multi-AZ in prod) + RDS Proxy	pgvector, RLS, familiar	Aurora PostgreSQL (read scaling / serverless v2)
Cache	ElastiCache (Valkey/Redis)	Rate limits, config cache	Skip in MVP (use Postgres)
Files	S3 (Intelligent-Tiering), S3 Object Lock for audit	Durable, cheap, presigned URLs	—
Queues	SQS Standard + DLQ (Celery supports SQS as broker)	Managed, cheap	Amazon MQ (RabbitMQ) for complex routing

Need	Service	Why	Alternatives (when)
Domain events & cron	EventBridge bus + EventBridge Scheduler	Replaces cron servers, retries/DLQ	Step Functions (stateful multi-step flows, e.g. account deletion)
LLM	Amazon Bedrock (Converse API, structured output, Guardrails, VPC endpoint)	Data stays in AWS, IAM, many models	Direct provider APIs; self-hosted on GPU EC2
OCR	PaddleOCR/VietOCR in the worker; Bedrock vision LLM for handwriting	Textract does not support Vietnamese	Textract for English documents
PII redaction	Amazon Comprehend (English PII) + rules/regex + custom Vietnamese NER	Comprehend doesn't cover Vietnamese fully	Open NER model (fine-tuned PhoBERT)
Email	Amazon SES	Cheap, DKIM/SPF	Resend
Secrets, keys	Secrets Manager, KMS (CMKs per purpose: phi, app, logs)	Rotation, audit	SSM Parameter Store (non-secret config)
Feature flags	AWS AppConfig	Safe rollout, rollback	Unleash
Container registry	ECR (scanned by Inspector)	ECS integration	GHCR
CI/CD	GitHub Actions + OIDC → IAM role; ECS deploy (rolling or blue/green with CodeDeploy)	No access keys	CodePipeline/CodeBuild
IaC	Terraform (state in S3 with native locking or DynamoDB lock)	Multi-cloud standard	AWS CDK

Need	Service	Why	Alternatives (when)
Observability	CloudWatch Logs/Metrics/Alarms, X-Ray via ADOT, CloudWatch RUM, Synthetics canaries	Native, low ops	Amazon Managed Grafana + Prometheus; Datadog
Security	GuardDuty (incl. Malware Protection for S3), Security Hub, Config, Inspector, IAM Access Analyzer, CloudTrail, Macie (PII in S3)	Continuous detection + compliance	—
Backup	AWS Backup (RDS, S3) + cross-account vault	Central, lockable	Manual snapshots
Analytics	S3 + Glue Data Catalog + Athena + Lake Formation; QuickSight	Serverless, pay per query	Redshift Serverless (large data)
Cost	AWS Budgets, Cost Explorer, Cost Anomaly Detection, tags `project/env/owner`	FinOps	—

19.4 AWS adoption by phase

Phase	Infrastructure	Learning goal
Foundation	Local Docker Compose (Postgres, MinIO, LocalStack for SQS/S3) + one AWS nonprod account: ECR, ECS Fargate, single-AZ RDS, S3, SQS, Cognito, CloudWatch	ECS, IAM roles, basic Terraform
Build MVP	Add Bedrock, EventBridge Scheduler, Secrets Manager, KMS CMKs, WAF, CloudFront	Serverless integration, data protection

Phase	Infrastructure	Learning goal
Hardening	Organizations + separate prod account, org-wide GuardDuty/Security Hub/Config/CloudTrail, cross-account AWS Backup, VPC endpoints, RDS Multi-AZ	Landing zone, compliance, DR
Operate / v1.5	Athena/Glue/Lake Formation, AppConfig, CloudWatch RUM/Synthetics, Cost Anomaly Detection	Data platform, FinOps
Optional	EKS + Argo CD, SageMaker for CV models	K8s, MLOps

19.5 Cost estimate (indicative)

The figures below are **rough, not checked against current prices** — build your own estimate in the AWS Pricing Calculator. The biggest items are usually NAT Gateway, RDS Multi-AZ, 24/7 Fargate, ALB and LLM tokens.

Setup	Main components	Estimated monthly cost
Minimal nonprod	1 small Fargate task, single-AZ RDS t4g.micro, no NAT (VPC endpoints + controlled public subnet) or off out of hours	Low (tens of USD)
Small prod (100 MAU)	2 API tasks + Spot worker, small Multi-AZ RDS, 1–2 NAT, ALB, WAF, CloudFront	Medium (100–250 USD)
Savings levers	Graviton (arm64), Fargate Spot for workers, Savings Plans once stable, nonprod off at night via Scheduler, S3 Intelligent-Tiering, LLM result caching	—

19.6 AWS Well-Architected mapping

Pillar	Applied in MediLens
Operational Excellence	IaC, runbooks, CI/CD, post-mortems, canaries

Pillar	Applied in MediLens
Security	Section 13, multi-account, KMS, VPC endpoints, GuardDuty
Reliability	Multi-AZ, SQS + DLQ, retries, AWS Backup, DR drills
Performance Efficiency	Fargate autoscaling, CloudFront, caching, Graviton
Cost Optimization	Spot, Budgets, tags, nonprod shutdown, Intelligent-Tiering
Sustainability	Graviton, scale-to-zero workers when idle

Run the free **Well-Architected Tool** in the console at the end of Hardening and store the report in `docs/architecture`.

20. Operations and maintenance

After launch, most of an enterprise product's effort goes into **operations**: keeping the system running, secure, cheap and correct. MediLens operations follow **a slimmed-down ITIL 4 + SRE**: a recurring calendar, a change process, an incident process and lifecycles for AI models and medical content.

20.1 Maintenance calendar

Frequency	Task	Role	Evidence
Daily (automatic)	Backup verification, nightly AI eval, data quality, cost anomaly, cron dead-man checks	SRE/DE/AI	Dashboard + alerts
Daily (10 min)	Review SLO dashboard, new Sentry errors, DLQ, GuardDuty findings	SRE	Daily note
Weekly	Merge Renovate dependency PRs, bug triage, AI correction-rate review, cost review	DevOps/QA/AI	Changelog

Frequency	Task	Role	Evidence
Weekly	Maintenance block (2 h): small tech debt, runbook updates	All	Tickets
Monthly	Patch base images + rebuild, rotate non-automatic secrets, IAM Access Analyzer review, retention cleanup, SLO/error-budget report	DevOps/Security/SRE	Monthly report
Monthly	Re-review due medical content, user feedback review	BA/Medical reviewer	Review list
Quarterly	DR drill (restore RDS + S3 into a temporary account), access review, incident tabletop, re-evaluate new models/providers, threat model update	SRE/Security/AI	Records
Quarterly	Re-run staging load tests, capacity review, Well-Architected review	SRE	Report
Every 6 months	Self-run ASVS pentest, DPIA and privacy policy review	Security	Report
Yearly	Major upgrades (PostgreSQL, Python, Node, Next.js), KMS key rotation check, architecture & roadmap review, domain renewal	All	ADR/Release

20.2 Change management

Type	Example	Process	Approval
Standard (pre-approved)	Merge a Renovate patch, deploy a CI-passed feature	Automated pipeline	None extra

Type	Example	Process	Approval
Normal	Schema migration, AI model/prompt change, new Unit, IAM change	Short RFC (purpose, risk, rollback, testing) in the PR	You as CAB with a checklist
Emergency	Patch a Critical vulnerability, SEV1 hotfix	Do first, write the RFC within 24 h	Recorded afterwards

Every production change has a ticket, PR, CI result, rollback plan, post-deploy smoke test and changelog entry. Freeze window: no Normal changes while the error budget is exhausted.

20.3 Incident and problem management

Level	Definition	Response	Example
SEV1	Data exposure, full outage	Immediately, updates every 30 min	Public PHI bucket, RDS down
SEV2	Core feature broken for many	≤ 1 hour	AI pipeline stopped, login failing
SEV3	Partial issue with workaround	≤ 1 business day	One Unit shows wrong colours
SEV4	Minor, cosmetic	Per sprint	Typo

Lifecycle: detect → classify → mitigate (rollback/feature flag/scale) → fix → **blameless post-mortem within 48 hours** (timeline, root cause via 5 Whys, dated action items) → **problem management**: recurring incidents become root-cause tickets. Public status page once beta starts.

20.4 Vulnerability and patch management

CVSS level	Fix SLA	Detection sources
Critical (9.0+)	24 hours (or a mitigation)	Inspector, Dependabot, Trivy, GuardDuty
High (7.0–8.9)	7 days	Same + ZAP

CVSS level	Fix SLA	Detection sources
Medium	30 days	Same
Low	90 days or documented risk acceptance	Same

20.5 FinOps (cost management)

- Mandatory tags `project`, `env`, `owner`, `cost-center` (checked by a Config rule).
- AWS Budgets alerts at 50/80/100% of the monthly budget; daily Cost Anomaly Detection.
- Unit costs: **USD per processed document**, **USD per active user**, **USD per 1,000 explanation questions** — tracked on a dashboard.
- Regular optimisation: right-size Fargate tasks from real CPU/RAM, Graviton, Spot for workers, nonprod off at night, review NAT data processing, cache LLM results, smaller models for simple steps (page classification).

20.6 AI model and prompt lifecycle (MLOps/LLMOps)

1. **Register:** each configuration (provider, model ID, prompt version, parameters) has an ID and a Model/Prompt Card.
2. **Pre-release evaluation:** golden set + red-team set; must not drop > 1 F1 point in any Unit, guardrails ≥ thresholds.
3. **Gradual rollout:** shadow mode (runs in parallel, not shown) → 10% canary (AppConfig) → 100%.
4. **Monitoring:** correction rate, confidence distribution, unmapped rate, latency, cost; drift alert when > 20% off last week.
5. **Rollback:** flip the flag back to the previous configuration in < 5 minutes.
6. **Retirement:** when a provider deprecates a model, plan the switch within 30 days; keep results reproducible (`model_version` stored on every `extraction_run`).

20.7 Medical content and Unit lifecycle

- KnowledgeArticle: draft → review (sources, date, language) → published → re-review after 12 months → archived.
- Unit config: changed via PR, automatic mapping tests, draft on staging, versioned publish; viewer cache invalidated on version change.

- Code catalogues (ICD-10, LOINC): updated when new editions appear, with tested data migrations.

20.8 Support and user feedback

Request type	Channel	Response SLA	Handling
Bug	In-app feedback (attaches traceId, never PHI)	2 business days	Bug ticket
Usage question	FAQ, email	2 business days	Update FAQ
Personal data request (access/delete/withdraw consent)	Settings + email	Acknowledge in 72 h, complete ≤ 30 days	DSR process, audit
Vulnerability report	`security.txt`, dedicated email	72 hours	Vulnerability disclosure policy

20.9 Mandatory operations documents

- [ ] Runbook for every alarm (symptoms, checks, fix, escalation)
- [ ] Runbooks for deploy, rollback, DB restore, S3 restore, secret rotation, DLQ handling, LLM provider switch
- [ ] On-call guide and contact list (vendors, authorities for data breaches)
- [ ] Service catalog: each service has owner, SLO, dashboard, repo, runbook
- [ ] Maintenance calendar and DR drill records

21. Role-based AI agents: the skill pack

Every project role has an **Anthropic-format Agent Skill** (a folder with `SKILL.md` carrying `name` + `description` frontmatter, plus `references/`, `assets/`, `scripts/`), and **`medilens-orchestrator`** routes work between them. In total: 14 skills, 175 files, packaged in `medilens-skills-full.zip` (shared in the chat); `dist/` holds one zip per skill for upload.

Every problem is routed to the owning role skill and moves on only after its gate



agent → skill flow · routing, execution, gate, hand-off

Claude keeps only each skill's `description` in context; the full instructions and reference files load only when a skill is used, so a large pack costs little context.

21.1 Skill list

Skill	Role	Triggered by	Main outputs	Scripts
`medilens-orchestrator`	Coordinator	Multi-role work, unclear owner, end-to-end plans	Routing, task briefs, playbooks PB-01…10, hand-offs	—

Skill	Role	Triggered by	Main outputs	Scripts
`medilens-business-analyst`	BA	Requirements, user stories, Gherkin, business rules, glossary, UAT	BRD, SRS, stories, BR catalog, traceability	`check_traceability.py`
`medilens-product-manager`	PO/PM	Priorities, sprints, roadmap, OKRs, risks, delays	Sprint plan, status, risk register, decision log	`rice_score.py`
`medilens-ux-ui-designer`	UX/UI	Screens, flows, viewer, wording, accessibility	Screen specs, design tokens, usability tests	—
`medilens-solution-architect`	Solution Architect	Technology choices, ADRs, NFRs, integration patterns	ADRs, NFRs, C4, design notes	—
`medilens-ai-ml-engineer`	AI/ML	OCR, CV, LLM, mapping, RAG, guardrails, eval, synthetic data	Schemas, prompts, eval reports, model cards	`eval_extraction.py`, `blur_check.py`, `fdi_parser.py`, `pii_redact.py`
`medilens-data-engineer`	DE	Data classification, pipelines, dbt, DQ, contracts, retention, deletion	Catalog, pipeline specs, contracts, DSR checklist	`pseudonymize.py`, `dq_check.py`
`medilens-backend-engineer`	BE	APIs, authorization, migrations, workers, outbox	Template-based code, authz tests, OpenAPI	—
`medilens-frontend-engineer`	FE	Pages, components, 3D/2D viewer, Unit plugins, performance	Code, plugins, normalised GLB/SVG assets	`check_glb_regions.py`
`medilens-cloud-devops-engineer`	Cloud/DevOps	AWS services, Terraform, CI/CD, deploys, cost	Terraform modules, CI/CD workflows, Dockerfile	—

Skill	Role	Triggered by	Main outputs	Scripts
`medilens-security-engineer`	Security	Threat models, reviews, OWASP, PDPL, incidents	Threat model, DPIA, vendor assessment, sign-off	`scan_log_for_pii.py`
`medilens-qa-engineer`	QA	Test strategy, test cases, automation, load tests	Test plan, cases, Playwright, k6	—
`medilens-qc-release-gatekeeper`	QC	GO/NO-GO, exploratory testing, medical content review, UAT	Readiness report, content review	`release_gate.py`
`medilens-sre-operations`	SRE/Maintain	SLOs, alarms, incidents, runbooks, DR, maintenance	Runbooks, post-mortems, DR records, ops report	`error_budget.py`

21.2 Anatomy of a skill (AI/ML example)

```
medilens-ai-ml-engineer/
├── SKILL.md                      # frontmatter + Workflow selector W1…W10 +
safety rules
├── references/
|   ├── project-context.md        # shared context (in every skill)
|   ├── pipeline-spec.md          # stage contracts, thresholds, failure
handling
|   ├── extraction-schemas.md     # JSON Schemas for dental / cardiology /
oncology
|   ├── prompt-library.md         # classify, extract, map, explain, judge
prompts
|   ├── evaluation-methodology.md
|   ├── guardrails-and-rag.md
|   ├── synthetic-data-recipe.md
|   └── medical-glossary-vi-en.md
├── assets/                       # eval report, model card, dataset card
└── scripts/                      # eval_extraction.py, blur_check.py,
fdi_parser.py, pii_redact.py
```

21.3 How the agent handles problems

You say	Skill used	What the agent does
“OCR misreads tooth numbers on blurry photos”	ai-ml-engineer (W6)	Samples 30 errors, tags causes, runs `blur_check.py` and `fdi_parser.py`, fixes the biggest bucket, re-runs eval
“Add the Nephrology–Urology department”	orchestrator → PB-02	BA → UX → AI (schema) → DE (catalogues) → FE (GLB/SVG) → BE (config) → QA → QC, gated at each step
“User A can see user B's document”	security-engineer (W8) → backend (W5)	Incident handling, immediate containment, fix with authz tests, assess notification duties
“Can we release 1.0?”	qc-release-gatekeeper (W1)	Gathers metrics, runs `release_gate.py`, writes a GO/NO-GO readiness report
“This month's AWS bill doubled”	cloud-devops-engineer (W6)	Cost Explorer by tag, finds NAT/Fargate/token drivers, proposes cuts
“The site keeps returning 500”	sre-operations (W3)	Declares SEV, rolls back/flips flag first, investigates by traceId, post-mortem

21.4 Installing and governing skills

- **Claude Code:** copy skill folders into the MediLens repo's `.claude/skills/` (commit to version them with code) or `~/.claude/skills/`; invoke directly with `/medilens-<role>`.
- **claude.ai / Claude app:** upload each zip from `dist/` in the Skills section.
- **Governance:** skills are "process as code" — change via PR, bump `metadata.version`, record why; when the real process changes (e.g. a new Unit or LLM provider), update `project-context.md` and repackage.
- **Limits:** the agent assists; GO/NO-GO, risk acceptance, legal and medical content remain your responsibility (and experts' once real data is involved).

Part III – Large-project review and extensions

Reviewing Parts I–II against the lifecycle of large healthcare software projects shows the plan is strong technically but missing **five areas**: project initiation and governance, **medical-device regulation (SaMD)**, **clinical safety**, **data governance and licensing**, and **post-launch operations (hypercare, support, project closure)**. Part III adds these areas and 9 roles, extends input to digital records, free-text descriptions and CT images, and sets a cost-aware model strategy. The new roadmap runs **52 weeks (26 sprints)** and replaces the 32-week roadmap in section 5; v1.0 still launches in week 32, and the day-level plan is in the attached Excel workbook.

22. Gap analysis: large-project lifecycle vs MediLens

The table compares the steps an enterprise-scale digital-health project normally requires with the current plan. *Status* shows whether the step already existed in Parts I–II or is added in Part III.

22.1 Lifecycle comparison

Stage	Required step / document in large projects	Status	What is added
Initiation	Project Charter, Business Case, Feasibility Study	Added	2-page charter (goals, scope, budget, risks, success criteria); business case with a do-nothing option
Initiation	Stakeholder register, communication plan, governance (steering committee)	Added	Steering = you + 1 mentor + 1 clinical advisor, meeting at the end of each phase; monthly report
Initiation	Intended Use Statement and medical-device (SaMD) classification	**Added – critical**	See 22.2; decision recorded as a Regulatory Decision Record

Stage	Required step / document in large projects	Status	What is added
Planning	Scope/schedule/cost baseline, WBS, baseline change control	Partly	Baseline lives in the Excel workbook; every baseline change goes through a change request
Planning	Quality Management Plan (lightweight QMS), document control	Added	Documents have an id, version and approver; CAPA process (corrective and preventive action)
Planning	Clinical Risk Management Plan, hazard log	**Added – critical**	ISO 14971 principles: identify hazards → assess → control → residual risk; signed by the Clinical Safety Officer
Planning	Procurement and vendor management, licence register (datasets, 3D assets, software, medical terminology)	Added	Licence register (section 25); open-source licence scanning in CI
Planning	Ethics review when using real patient data from a facility	Added	Data sharing agreement + the partner's ethics committee approval before receiving data
Planning	Responsible AI: AI impact assessment, model risk tiering, transparency	Added	AI impact assessment for every new model or use case; bias checks by group (age, sex, image quality, handwriting)
Design	Design controls: user needs → design input → design output → verification → validation	Partly	Extended traceability: BR/FR → spec → code → test → usability/clinical validation (22.3)
Design	Usability engineering (IEC 62366, formative and summative)	Partly	Summative usability test in Hardening with use-error scenarios

Stage	Required step / document in large projects	Status	What is added
Design	Interoperability and terminology (FHIR, DICOM, ICD-10, LOINC, SNOMED CT)	Partly	Clinical Informaticist role; terminology licences; digital record import (section 26)
Build	Secure SDLC, architecture, code, tests	Existing	Secure data-service design added (section 24)
Verification	System V&V, clinical/expert validation (clinician agreement on mapping and content)	Added	Validation study with ≥ 2 clinicians on 100 synthetic records; agreement measured
Verification	Independent pentest, vulnerability disclosure programme	Partly	External pentest before any real data; `security.txt` + VDP
Release	Release management, Go/No-Go, release notes	Existing	Regulatory and clinical-safety checks added to the release gate
Release	2–4 weeks of hypercare, L1/L2/L3 support, user documentation, training	Added	Support & Customer Success and Technical Writer roles
Release	Go-to-market: positioning, marketing-claims control (no diagnostic claims)	Added	Claims review approved by Regulatory
Operate	Service management, SLA, BCP, continuous improvement	Existing	Business Continuity Plan and post-market surveillance (tracking clinical-safety incidents)
Closure	Lessons learned, acceptance, handover, decommissioning and data destruction plan	Added	Closure report per phase; decommission plan (user export, secure deletion, notices)

22.2 Medical-device regulation (SaMD) — decide early

In Vietnam, the definition of a medical device includes **software** used for medical purposes such as diagnosis or monitoring (Pure Global). The base text is Decree 98/2021/ND-CP, amended by Decree 07/2023 and Decree 04/2025. Classes A–D follow Circular 05/2022 and later amendments. Classification depends on the **intended use**. Therefore:

1. **Write the Intended Use Statement during Discovery.** Example: *"MediLens helps users review and understand information already written in their own health records, through illustrations and sourced educational content. MediLens does not diagnose, does not recommend treatment and does not analyse images to detect disease."*
2. **Keep v1 within illustration/education:** no automatic lesion detection on images, no risk scores, no treatment suggestions. Any feature beyond this line (for example CV that detects lesions on CT) needs a new assessment.
3. **Write a Regulatory Decision Record:** intended use, reasoning for why it is (not) a medical device, legal references, next review date. Before commercialising or adding CT, ask a lawyer or medical-device registration expert and check the Ministry of Health portal.
4. **Control claims:** landing page, store listing and LLM output must not use words such as "diagnose", "detect disease" or "replaces your doctor".
5. **International references for learning:** the IMDRF SaMD framework, IEC 62304 (medical software lifecycle), ISO 14971 (risk management), IEC 62366 (usability), ISO 13485 (QMS). A personal project does not need certification, but working in their spirit is how large projects operate.

Electronic medical records context: Circular 13/2025/TT-BYT (effective 21 July 2025) requires hospitals to deploy EMRs by 30 September 2025 and other facilities by 31 December 2026. It does not regulate third-party apps or patient APIs, so direct integration depends on each facility. MediLens therefore works with copies of records the user already holds (photos, PDFs, exported files).

22.3 Design controls applied to MediLens

1. **User needs** (personas, interviews) → `UN-xxx`
2. **Design inputs** (BR, FR, NFR, SEC, clinical safety requirements `CS-xxx`) → BRD/SRS
3. **Design outputs** (architecture, API, schema, prompts, UI spec, code)
4. **Verification** (unit/integration/E2E/AI eval: "did we build it to spec?")

5. **Validation** (summative usability, clinician review: "does it meet the need safely?")
6. **Design transfer / release** (release gate, release file)
7. **Post-market** (feedback, safety incidents, hazard-log updates)

The traceability matrix gains `UN`, `CS` (clinical safety requirement) and `VAL` (validation evidence) columns; the BA skill's `check_traceability.py` is reused.

22.4 New gates added to the roadmap

Gate	When	Criteria
G0-R (Regulatory)	End of Discovery	Intended Use + Regulatory Decision Record approved
G1-D (Data licence)	Before training/fine-tuning any model	Every dataset licensed for its use; Dataset Card complete
G2-CS (Clinical safety)	Before beta	No "unacceptable" risk left in the hazard log; Clinical Safety Case signed
G3-P (External pentest)	Before receiving real data	No open High/Critical; pentest report archived
G5-H (Hypercare exit)	4 weeks after launch	No new S1/S2 for 2 weeks, SLOs met, support backlog stable

Section 29 adds G5 (release v1.5) and G6 (release v2.0), and re-runs G0-R before CT work and G1-D before fine-tuning. Release checks inside G3, G5 and G6 are named RG-01…RG-21 so they never clash with phase gates.

23. Missing roles and how they are added

Large healthcare projects always include people who are **not engineers** yet can block a release: a clinician accountable for clinical safety, someone accountable for regulation and privacy, and someone who governs data. MediLens adds **9 roles**, each with a new skill in the skill pack (23 skills in total). For the three roles marked ★ you still play the role to learn, but **a real person is required** before real data or a public launch.

23.1 Added roles

Role	Why large projects need it	Main responsibilities	Phases	Skill
★ Clinical Safety Officer / clinical advisor	Wrong medical content can cause harm; a qualified person must be accountable	Hazard log, Clinical Safety Case, review of anatomical mapping and content, clinical validation, post-launch safety monitoring	Discovery → Operate	`medilens-clinical-safety-officer`
★ Regulatory, Privacy & Legal (incl. DPO)	Medical-device rules, personal data law, contracts, licences	Intended use, Regulatory Decision Record, DPIA/OTIA, consent, terms, claims control, AI governance policy	Discovery → Operate	`medilens-regulatory-privacy-officer`
Clinical Informaticist	Health-data standards decide interoperability and coding quality	FHIR model, ICD-10/LOINC/SNOMED mapping, DICOM, digital record import, terminology licences	Foundation → v2	`medilens-clinical-informaticist`
★ Data Curator & Annotation Lead	A model is only as good as its data; medical data needs licences and expert labels	Data sourcing, licence register, annotation guidelines, managing annotators (medical students), label QA, Dataset Cards	Discovery → Operate	`medilens-data-curator`
MLOps Engineer	Models must be served, monitored and cost-optimised as a system	Model registry, tiered model router, cache, batch, GPU serving, cost dashboard, drift	Build → Operate	`medilens-mlops-engineer`

Role	Why large projects need it	Main responsibilities	Phases	Skill
Medical Imaging Engineer	CT/DICOM is a specialist domain	DICOM intake, de-identification, slice viewer, segmentation, meshing, registration to the template	v1.5 → v2	`medilens-medical-imaging-engineer`
3D Medical Artist	Anatomy models must be correct, calm, light and licensed	Build/adjust meshes, name by region code, anatomical variants, implant library, LOD	Foundation → v2	`medilens-3d-medical-artist`
Technical Writer & Localization	Wrong documents or medical translations mislead users	User guide, FAQ, API docs, release notes, glossary-driven VI/EN translation, terminology checks	Build → Operate	`medilens-technical-writer`
Support & Customer Success	After launch someone must receive feedback and incidents	L1/L2/L3 support model, hypercare, knowledge base, triage of clinical-safety feedback, satisfaction	Beta → Operate	`medilens-support-success`

23.2 Other enterprise roles (merged into existing roles)

Enterprise role	Merged into	Note
Scrum Master / Delivery Manager	Product Manager	Protects sprint rituals, removes blockers
Release Manager	QC Release Gatekeeper	Release calendar, Go/No-Go
DBA	Backend + Data Engineer	Tuning, backup, migration
FinOps Analyst	Cloud/DevOps + MLOps	Infrastructure cost and token cost

Enterprise role	Merged into	Note
Penetration Tester	Security (self) + external tester before G3-P	Do not certify yourself at the final gate
Accessibility Specialist	UX/UI + QA	Screen-reader testing
Ethics / Responsible AI	Regulatory & Privacy + Clinical Safety	AI impact assessment

23.3 When real people are needed

Milestone	Person	Suggested arrangement
End of Discovery	Clinical advisor (1–2 hours/month)	Mentor or a friend in medicine; signs the Clinical Safety Case at G2-CS
Before real data	Lawyer / data-protection expert	One-off advice on DPIA, consent, OTIA, intended use
Before large-scale labelling	2–3 medical students	Short contract, NDA, synthetic or de-identified data only
Before G3-P	Independent pentester	Fixed-scope engagement or a small bug bounty

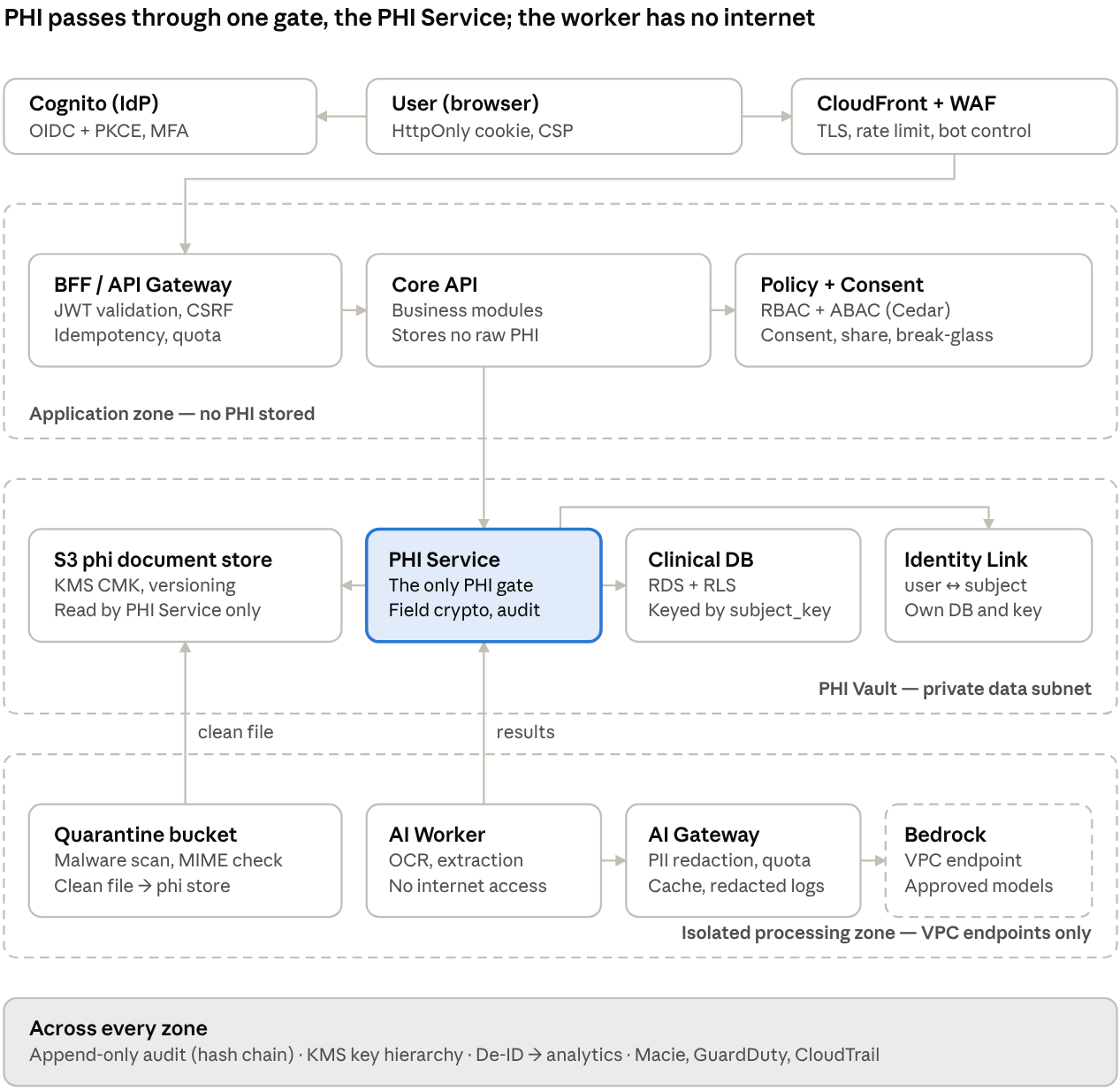
23.4 RACI for the new roles

Decision	CSO	Reg/Priv	Clin. Inf.	Data Curator	MLOps	Imaging	3D Artist	Writer	Support
Publishing medical content	A	C	C	I	I	I	I	R	I
Anatomical region mapping	A	I	R	C	I	C	R	I	I
Using a dataset for training	C	A	C	R	C	C	I	I	I
Changing model/provider	C	C	I	C	R	I	I	I	I
Adding CT input	A	A	C	C	C	R	C	I	I
Public release	A	A	I	I	C	I	I	C	C

Decision	CSO	Reg/Priv	Clin. Inf.	Data Curator	MLOps	Imaging	3D Artist	Writer	Support
Safety-related feedback	A	C	I	I	I	I	I	I	R

24. Backend and service design that protects health data

Core principle: **only one service — the PHI Service — may read or write health data**, user identity is **separated** from clinical data by a pseudonymous `subject_key`, each record has **its own encryption key** so deletion works by crypto-shredding, and the AI processing zone has **no internet**, calling LLMs only through the AI Gateway. This still fits the modular monolith (ADR-001): the PHI Service and the AI Gateway run separately because they are security boundaries, not because they need to scale separately.



trust zones and the path of health data

The Core API never queries the Clinical DB or the `phi` S3 bucket directly; every request goes through the PHI Service, where policy checks, field decryption and audit happen in one place.

24.1 Services and controls

Service	Responsibility	Data class	Storage	Called by	Key controls
BFF / API Gateway	Authentication, sessions, CSRF, rate limits, idempotency	C2	Redis (no PHI)	Browser	JWT audience/issuer checks, HttpOnly cookies, WAF in front
Core API	Non-PHI business logic: Unit config, knowledge, sharing metadata, notifications	C1–C2	RDS schema `core`	BFF	No IAM/DB rights to PHI
Policy & Consent	Access decisions: role + ownership + share + consent + break-glass	C2	RDS schema `policy`	Core API, PHI Service	Policies in Cedar (Amazon Verified Permissions or the Cedar library in-service), with tests
PHI Service	The only gate that reads/writes C4: documents, OCR, extraction, findings, observations	C4	Clinical DB, S3 `phi`	Core API, AI Worker (own IAM role)	Calls Policy per request; field encryption; audit; returns the minimum fields
Identity Link	Maps `user_id` ↔ `subject_key`	C3	Separate DB + separate CMK	PHI Service only	Separates identity from clinical data; a Clinical DB leak does not reveal identities
Ingestion (quarantine)	Receives files by presigned URL, malware scan, MIME check, moves clean files to S3 `phi`	C4	S3 quarantine (24 h TTL)	Browser (conditional PUT)	Separate bucket nobody reads except the scanner; GuardDuty Malware Protection for S3
AI Worker	OCR, extraction, mapping	C4 (in memory)	Nothing long-lived; results written through the PHI Service	SQS	Subnet without an internet route; VPC endpoints only to S3, SQS, AI Gateway

Service	Responsibility	Data class	Storage	Called by	Key controls
AI Gateway	Every LLM call: PII redaction, prompt policy, quota, cache, model routing, redacted logs, cost metering	C4 → C2	Hash-keyed cache (KMS), 30-day logs	AI Worker, Explain service	Model allowlist; guardrails; circuit breaker; IAM deny on direct Bedrock calls elsewhere
De-ID service	Exports pseudonymised/de-identified data to analytics and datasets	C4 → C2/C0	S3 analytics	DE pipeline	HMAC, C3 fields dropped, k-anonymity checks
Audit service	Append-only access events with a tamper-evident hash chain	C2	RDS + S3 Object Lock	Every service	No delete/update API; each record carries the previous record's hash

24.2 Key hierarchy

Key	Type	Protects	Rotation	Note
`kms/phi`	KMS CMK	S3 `phi`, Clinical DB, PHI backups	Yearly (automatic)	PHI Service and AI Worker (S3 decrypt) only
`kms/identity`	KMS CMK	Identity Link DB	Yearly	PHI Service only
Per-record DEK	Data key (envelope) from `kms/phi`	Sensitive fields and OCR text of one record	On demand	Deleting the DEK makes the record unrecoverable (crypto-shredding)
`kms/app`	KMS CMK	Secrets, cache, SQS	Yearly	—
`kms/logs`	KMS CMK	CloudWatch Logs, audit S3	Yearly	Log-archive account
Analytics HMAC key	Secret	Pseudonymisation	On suspected exposure	Read by the De-ID service only

24.3 Data access matrix

Principal	Own C4	Shared C4	Other people's C4	C3	Analytics (C2)
Patient	Read/write/delete	—	No	Own	No
Caregiver	Dependant's records	—	No	Own	No
Clinician	—	Read within scope + expiry	No	Sharer's name	No
Admin	—	—	Break-glass only (1 h, reason, alert)	As needed for duty	Dashboards
AI Worker (machine)	Documents assigned by job	—	No	No	No
Data Engineer	No	No	No	No	Read/write
AI/ML Engineer	No (synthetic or consented + de-identified only)	No	No	No	Read

24.4 Main secure flows

1. **Upload:** BFF issues a presigned PUT to the quarantine bucket (5 minutes, size/type limits) → GuardDuty Malware Protection scans → a Lambda/worker checks magic bytes and renders PDF to images → the PHI Service records metadata and moves the object to S3 `phi` → the quarantine copy is deleted.
2. **Reading findings for the viewer:** Core API calls the PHI Service with the principal → Policy (Cedar) returns permit/deny with a reason → the PHI Service reads by `subject_key`, decrypts only needed fields, writes audit → returns a minimal DTO.
3. **Calling an LLM:** the worker sends text to the AI Gateway → PII redaction → quota/budget check → cache lookup → tier selection (section 28) → Bedrock through a VPC endpoint → JSON Schema validation → result returned, redacted log written.
4. **Analytics:** the De-ID service reads through the PHI Service with its own role, HMACs identifiers, drops C3, writes Bronze; nobody holds both the HMAC key and analytics read access.
5. **Account deletion:** Step Functions revokes sessions → deletes the records' DEKs → deletes S3 objects (all versions) → deletes the Identity Link row → deletes analytics rows by pseudo id → confirms.

6. **Emergency admin:** no SSH; access through SSM Session Manager with session recording; break-glass grants temporary rights through IAM Identity Center and expires automatically.

24.5 Options by maturity level

Level	When	Design
A — learning MVP	Synthetic data only	PHI Service is a separate module in the monolith with its own DB schema and role; RLS; AI Gateway is a shared library
B — beta with real data	Before G3-P	PHI Service and AI Gateway become separate ECS services with their own IAM roles; separate Identity Link DB; per-record DEK; worker subnet without internet
C — enterprise	With hospital partners	Separate AWS account for the PHI Vault, PrivateLink between accounts; Verified Permissions; periodic Macie scans; SIEM; yearly pentest

24.6 Additional requirements (SEC-AR-xx)

Code	Requirement	Strength
SEC-AR-01	Only the PHI Service has IAM/DB access to the Clinical DB and S3 `phi`; verified by IAM Access Analyzer and tests	MUST
SEC-AR-02	The Clinical DB holds no email, name or phone; only `subject_key`	MUST
SEC-AR-03	The worker subnet has no route to a NAT/Internet Gateway	MUST
SEC-AR-04	Every LLM call goes through the AI Gateway; IAM denies `bedrock:InvokeModel*` to every other role	MUST
SEC-AR-05	Hash-chained audit log with a daily integrity check	SHOULD

Code	Requirement	Strength
SEC-AR-06	Per-record DEK; account deletion by crypto-shredding	SHOULD (MUST from level B)
SEC-AR-07	Access policies as code (Cedar) with tests for BR-006, BR-030, BR-031	MUST
SEC-AR-08	DICOM de-identified (PS3.15) before any processing; the original can only be re-downloaded by its owner	MUST (v2)
SEC-AR-09	Imported PDF/DOCX/ZIP parsed only in the sandboxed worker after a malware scan; macros, JavaScript actions, external relationships and zip bombs rejected	MUST

25. Data strategy: sourcing, licences, how much is enough, maintenance

Medical data differs from ordinary data in three ways: it is **sensitive** (needs consent and de-identification), **licence-bound** (many datasets forbid commercial use or redistribution), and **needs expertise to label**. MediLens follows four tiers from lowest to highest risk: **(1) self-generated synthetic data → (2) public data with a suitable licence → (3) opt-in donations from users → (4) data from partner healthcare facilities**. Move up a tier only after passing the previous tier's gate.

25.1 What data each AI component needs

Component	Data needed	Labels	Minimum evaluation set	Training/fine-tuning set (if any)
Image quality check	Good/bad document photos	good/bad + reason	300 images	No training (thresholds)
Page classification	Record pages in 9 types	page type	30 per type (≈ 270)	200–500 per type if training a classifier

Component	Data needed	Labels	Minimum evaluation set	Training/fine-tuning set (if any)
Vietnamese OCR	Printed + handwritten lines	ground-truth text	2,000 lines	Only to fine-tune VietOCR: 10k+ lines
Field extraction	All record types per Unit	JSON per schema	200 per Unit (golden)	1,000–3,000 documents per Unit to distil a small model
Code normalisation	(phrase → ICD-10/LOINC) pairs	correct code	500 pairs	Dictionary + embeddings, little training
Anatomical mapping	Location phrase → region_code	region + side	300 phrases per Unit	Rules + LLM, no training
RAG explanations	Approved articles + questions	reference answers, sources	200 questions + 100 red-team	No training
Text descriptions	Symptom descriptions VI/EN	entities + locations	300 sentences	No training in v1
CT (v2)	CT with organ segmentation	organ masks	30–50 reference cases	Off-the-shelf model (TotalSegmentator)
Teeth on X-ray (v1.5+)	Panoramic images with tooth numbers	bbox + FDI number	100 images	700+ images (DENTEX scale as reference)

25.2 Data sources and licences (checked)

Source	Content	Licence / conditions	Usable for MediLens?
Self-generated synthetic	Fake records from each Unit's templates	Yours	Yes — main source for development and the golden set

Source	Content	Licence / conditions	Usable for MediLens?
VinDr-CXR (Vietnam)	Chest X-rays annotated by radiologists	PhysioNet Credentialed Health Data License 1.5.0; credentialed account, CITI course, signed DUA	Research/learning under the DUA only; no redistribution; not in a commercial product unless the DUA allows
MIMIC-IV-Note and other credentialed PhysioNet sets	English clinical notes	Similar credentialed DUA	As above; internal learning/evaluation only
TCIA collections (e.g., LIDC-IDRI for lung CT)	CT, pathology, cancer	Mostly CC BY 3.0/4.0 including commercial use; some collections restricted; head-and-neck sets with facial-reconstruction risk moved to restricted licences (TCIA Data Usage Policy)	Yes, with correct citation; check each collection
Medical Segmentation Decathlon	10 tasks: 6 CT (liver, lung, pancreas, colon, hepatic vessels, spleen), 4 MRI	CC BY-SA 4.0 (commercial use allowed; derivatives share alike)	Yes — mind share-alike for derived data
TotalSegmentator on NCI IDC	Automatic anatomical segmentation of CT (mostly NLST)	CC BY 4.0	Yes — reference for the CT pipeline
DENTEX	Panoramic dental X-rays with tooth numbering and pathology	CC BY-NC-SA 4.0	Non-commercial only (learning, evaluation); not in a commercial release

Source	Content	Licence / conditions	Usable for MediLens?
MTSamples (Kaggle copy)	~5,000 English clinical transcriptions	The Kaggle copy says CC0 but was scraped from mtsamples.com; original terms unclear (clinspacy docs)	Internal learning/evaluation only; never in the product
ViMQ	Vietnamese medical questions with intent and entity labels	No published licence	Contact the authors before use
ICD-10 Vietnamese edition (Ministry of Health)	Disease code list	Government document	Yes — load as master data
LOINC	Lab codes	Free under LOINC terms, registration required	Yes
SNOMED CT	Clinical terminology	Non-member countries apply via MLDS; fee by World Bank class, possibly waived (SNOMED International)	Only with a licence; v1 uses ICD-10 + own dictionary
BodyParts3D / Z-Anatomy	3D anatomy models	CC BY-SA	Yes, with attribution; modified assets share alike

Reading licences: `NC` = no commercial use; `SA` = derivatives under the same licence; `DUA` = no redistribution, usually purpose-limited. Check separately the **right to train models** and the **right to use a trained model in the product**. Every dataset enters the **License Register** before download (gate G1-D).

25.3 Collecting real data (when ready)

Channel	How	Mandatory conditions
User donations (opt-in)	After confirming results, the user chooses "Donate my de-identified record to improve MediLens"	Separate consent, clear purpose, withdrawable; automatic de-identification + spot checks; used only in dataset versions after the consent date
Clinicians/medical students write fictional cases	Commission 200–500 realistic fictional records per Unit	Rights-assignment contract, NDA, not based on real patients
Partner facilities	Receive de-identified record sets for research	Data sharing agreement, the partner's ethics committee approval, DPIA, the facility's EMR rules, OTIA if processed outside Vietnam
Labelling by medical students	Label synthetic/de-identified data	Annotation guideline, NDA, transparent pay, clinician reviews 10–20%

De-identification: text uses `pii_redact.py` + NER with random human checks; the 18 HIPAA Safe Harbor identifier types serve as a checklist. DICOM uses the PS3.15 confidentiality profile, removes identifying tags and checks for burned-in text with OCR.

25.4 How much is enough?

- **Evaluation set:** size by confidence interval $n = z^2 \cdot p(1-p) / e^2$. With expected accuracy p = 0.85, margin e = ±5% and 95% confidence (z = 1.96): n ≈ 196 → **about 200 samples per reported group** (each Unit, each main document type). For ±3% you need about 545.
- **Training set:** decide with a **learning curve**. Train on 10%, 25%, 50% and 100% of the data; if the score still rises clearly at 100%, collect more; if it has plateaued, stop. The numbers in 25.1 are only starting points.
- **Coverage beats volume:** each group (printed/handwritten, good/bad photo, each form type, VI/EN) needs ≥ 30 samples in the golden set to reveal bias.

- **Label quality:** Cohen's kappa ⩾ 0.8 between two annotators; a clinician adjudicates disagreements.

25.5 Keeping data usable for models

Activity	Cadence	Details
Dataset registry	Every version	Each dataset has an id, immutable version, checksum, Dataset Card, licence, source, creation date and the models that used it
License register	On new sources; quarterly review	Licence still valid, citation terms, commercial limits
Golden-set refresh	Quarterly	Add 10–20% new samples reflecting real errors; keep a fixed core for comparison over time
Drift monitoring	Weekly	Document-type mix, image quality, user correction rate; > 20% shift → collect more data
Consent withdrawal	On event	Excluded from later dataset versions; the next model trains without it; lineage recorded
Relabelling	On schema change	Label conversion script + sample check; schema version tied to dataset version
Retention & deletion	Per table 14.3	Synthetic datasets kept long term; donated data kept for the published period
Data → model lineage	Every training run or prompt change	Model Card records dataset versions; trace which model used which data

25.6 Gate G1-D checklist (before training/fine-tuning)

- Every source is in the License Register and its licence allows this use (training, commercial if needed)
- Complete Dataset Card; checksum; immutable version

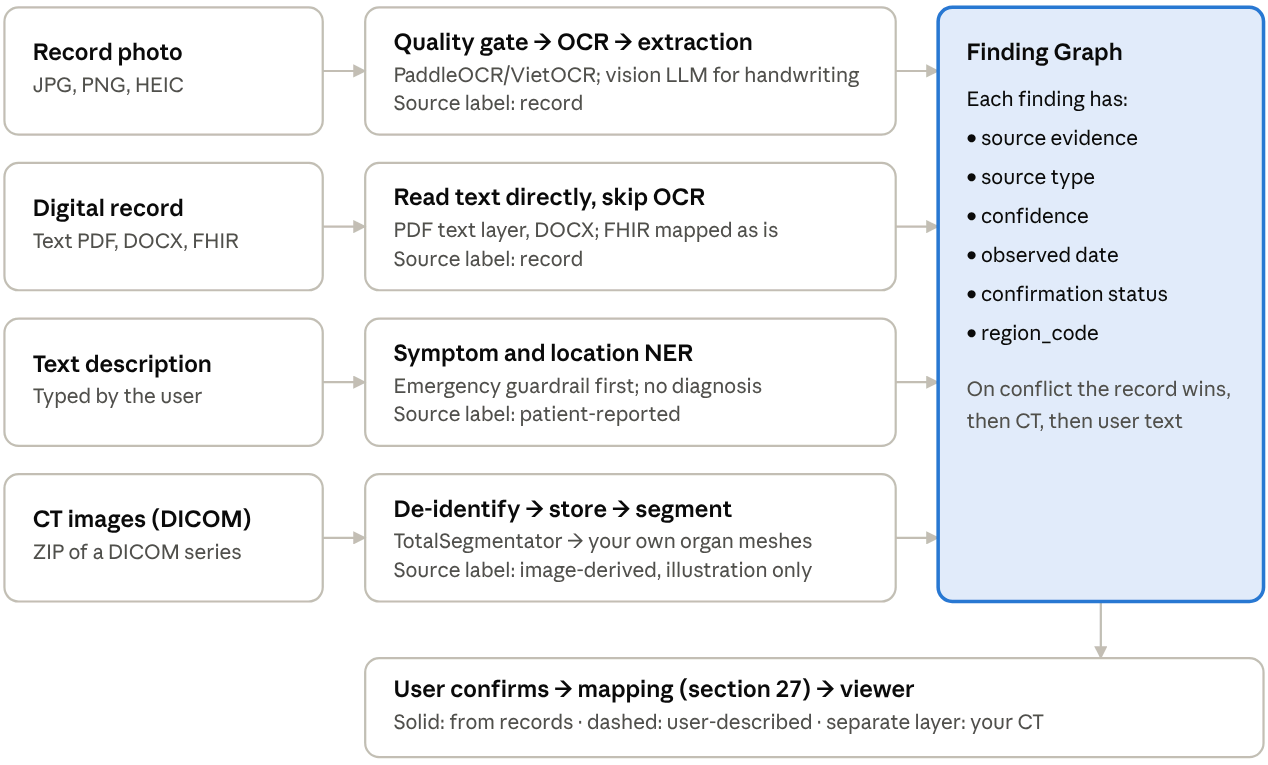
- No real data without consent; de-identification sample checked
- Enough coverage per group; labels reach kappa ≥ 0.8
- Golden set separate, no leakage into training (hash-based duplicate check)
- Signed by Regulatory & Privacy and the Data Curator

The data-curator skill automates parts of this checklist: `license_register_check.py`, `sample_size.py`, `kappa.py` and `leakage_check.py`.

26. Multimodal input: photos, digital records, text descriptions, CT

Each input type has its own pipeline but all feed **one shared finding model**. Every finding carries a *source type* (from a record / described by the user / derived from a CT image), *source evidence*, *confidence* and *confirmation status*. The viewer shows source types differently so users never confuse "what I described" with "what my doctor wrote".

Four input types, one model: every finding records its source



4 input types · separate pipelines · one shared finding model

When sources conflict (for example the user describes left-sided pain but the record says right), the record wins and both are shown with a warning: "information does not match — ask your doctor".

26.1 Specification per input type

Input	Format & limits	Processing	Result	Display	Main risks
Record photo	JPG/PNG/HEIC, ⩽ 20 MB per image, ⩽ 30 pages	Pipeline in section 6.4	`record` finding with bounding box	Solid style, severity colour	OCR errors, handwriting
Digital record	Text-layer PDF, DOCX, FHIR Bundle (JSON) exported from a hospital portal (if available)	Detect text layer → extract text directly (pdfplumber, python-docx); scanned PDF → photo pipeline; FHIR → map Condition/Observation directly, no LLM	`record` finding, higher confidence	Same as photo	PDFs with scripts/macros → rendered to image/text in a sandbox; FHIR with the wrong profile
Text description	Up to 2,000 characters, VI/EN	Emergency guardrail first → NER (symptoms, body locations, time) → region mapping	`patient_reported` finding, no diagnostic code	Dashed style, "you described" icon, no severity colour	Turning into a symptom checker — forbidden; only show location and suggest questions for the doctor
CT images (DICOM)	ZIP of one or a few CT series, ⩽ 1 GB (configurable), CT modality only in v2	DICOM checks → de-identification (PS3.15, tag removal, burned-in text check) → store → organ segmentation → meshing → alignment to the template	The user's own organ meshes + lesion locations **from the radiology report**	Separate "your CT" layer; slice viewer	GPU cost, large files, risk of becoming SaMD

26.2 Shared finding model (example)

```
{
  "findingId": "0191…",
  "subjectKey": "sk_…",
  "sourceType": "record | patient_reported | image_derived",
  "evidence": [{"kind": "document", "documentId": "…", "page": 2, "bbox":
[120, 340, 610, 372], "quote": "K phổi phải T2N1M0"}],
  "condition": {"icd10": "C34.1", "text": "Cancer of the right upper lobe"},
  "region": {"code": "lung_r_upper", "laterality": "right",
"mappingConfidence": 0.92, "precision": "segment"},
  "severity": "severe",
  "observedAt": "2026-03-02",
  "status": "confirmed",
  "organStatus": "present"
}
```

Rule: `image_derived` describes **anatomy** only (what your organs look like) and never creates a pathological finding by itself. Lesion locations on CT come from the accompanying radiology report (for example "12 mm nodule in segment S6") and are placed on the matching organ mesh.

26.3 CT pipeline in detail (v2)

1. **Upload:** ZIP via presigned multipart upload into quarantine; limits on size and number of files.
2. **Checks:** read DICOM headers (pydicom), accept Modality = CT only, enough contiguous slices, slice thickness ⩽ 5 mm.
3. **De-identification:** apply the Basic Application Confidentiality Profile (remove/replace name, birth date, patient id, institution…), remap UIDs, OCR check for burned-in text; keep the encrypted original only for the user to re-download.
4. **Storage:** S3 `phi` (Singapore). Note: AWS HealthImaging is currently available only in us-east-1, us-west-2, ap-southeast-2 (Sydney), eu-west-1 and eu-west-2 — **not in Singapore**.
5. **Segmentation:** TotalSegmentator on GPU on demand (AWS Batch with GPU Spot or SageMaker Asynchronous Inference, scaled to zero when idle); CPU as a slower fallback.
6. **Meshing:** marching cubes on masks → smoothing → decimation → GLB, meshes named by region_code via a TotalSegmentator label → region_code table.

7. **Alignment:** see section 27 (bring your meshes into the template's coordinate system).
8. **Display:** slice viewer (OHIF/Cornerstone3D) + a 3D "your CT" layer, clearly labelled "anatomical illustration, not a radiology result".

26.4 Implementation options per input type

Input	Option A (simple, cheap)	Option B (balanced — recommended)	Option C (enterprise)
Photo	Vision LLM reads the image directly	Local OCR + LLM extraction from text; vision LLM only for handwritten regions	Add a layout model (LayoutLM) fine-tuned on Vietnamese data
Digital record	Text-layer PDF only	PDF + DOCX + FHIR Bundle	Direct hospital-portal integration under an agreement (SMART on FHIR)
Text description	Store and show the raw text in the timeline	NER + region mapping, `patient_reported` label	Add voice input (speech-to-text)
CT	Slice viewer + radiology report only	Organ segmentation + your meshes + locations from the report	DICOMweb (Orthanc) + dedicated imaging storage (HealthImaging if another region is acceptable and an OTIA is done)

26.5 New business rules

Code	Rule
BR-040	A `patient_reported` finding gets no ICD-10 code and no severity colour, and always shows a "you described" label.
BR-041	A text description with emergency signs shows 115 before any other content (like BR-023).

Code	Rule
BR-042	CT images are used only to build illustrative anatomy; no automatic detection or measurement of lesions (stays outside SaMD scope — section 22.2).
BR-043	DICOM is de-identified before any processing step; only the user can re-download the original.
BR-044	When sources conflict, `record` > `image_derived` > `patient_reported`, with a mismatch warning.
BR-045	Text-layer digital records are extracted directly without OCR; valid FHIR is mapped without the LLM.

27. Handling gaps between the patient's body and the template model

The 3D template is an "average" body. Real users differ in size, sex and age, may lack organs after surgery, carry implants or have anatomical variants. Guiding principle: **never draw a wrong location with confidence**. MediLens picks the closest template, marks organ status from the record, shows how precise each location is, and uses the user's own body when a CT is available.

27.1 Types of mismatch and how to handle them

#	Mismatch	Example	Detected from	Handling	Display
1	Build, sex, age	Children, very tall/short people, male/female	Profile (birth year, biological sex, height, weight)	Template per group (adult male/female, 3 child age bands); scale by height/weight (SMPL-style shape parameters for the surface, organs scaled by landmarks)	No warning needed

#	Mismatch	Example	Detected from	Handling	Display
2	Removed or absent organ	Nephrectomy, cholecystectomy, amputation, extracted tooth	Record mentions surgery/tooth loss → `organStatus = removed/absent`	Hide the mesh or show a ghost with the label "removed (per record, date…)"	Faint outline + label
3	Implanted devices	Pacemaker, stent, joint replacement, dental implant, plates and screws	Procedure records	3D icon library for devices, attached to the matching region	Distinct icon, legend
4	Anatomical variant	Situs inversus, horseshoe kidney, supernumerary tooth	Explicit in the record	Use a variant mesh if one exists; otherwise keep the template + warning "your anatomy differs from the model" and prefer the 2D diagram	Warning banner
5	Deformation from disease	Enlarged liver, large tumour, effusion	Sizes in the record (`size_mm`)	Do not deform the organ mesh; draw the lesion marker to scale against the standard organ size, with the measurement label	Scaled marker + measurement
6	Location detail mismatch	Record says "right lung" but the mesh is split by lobe; or a liver segment while the mesh only has the liver	Mapping returns `precision`	Hierarchical region tree: tint the parent faintly when only the parent is known; the child when exact	"Approximate location" label
7	Unknown side	"Kidney" without left/right	`laterality = unknown`	Tint both sides faintly and ask the user to confirm on the review screen	Warning + question

#	Mismatch	Example	Detected from	Handling	Display
8	User's CT differs from the template	Organs at different position/size	Meshes from CT	Registration (27.3) or a separate "your CT" mode	Separate layer, compare switch
9	Change over time	Before/after surgery	Timeline encounters	Organ status per point in time	Timeline slider

27.2 Region tree and location precision

- Each `region_code` has a parent: `body → thorax → lung_r → lung_r_upper → lung_r_upper_apical`; meshes are needed only at some levels, the rest is derived from the tree.
- Mapping returns `precision ∈ {exact, segment, organ, system, body_part}` and `mappingConfidence`.
- Display rules: `exact/segment` → tint the child region; `organ` → tint the whole organ; `system/body_part` → faint large area with "approximate location"; confidence < 0.6 → `unmapped` (BR-015).
- Each node carries a standard reference code (FMA, or SNOMED CT body structure once licensed) under the Clinical Informaticist's control.

27.3 Registration when a CT exists (v2)

1. Segmentation yields masks of the user's organs and bones.
2. Compute **anatomical landmarks**: vertebral centroids, sternum, iliac crests, organ centroids.
3. **Rigid + affine** alignment from the user's landmarks to the template's (least squares) to share one coordinate system and orientation.
4. Optional **non-rigid** alignment (thin-plate spline on landmarks) so the template hugs the user's body when comparing.
5. **Prefer the user's own organ meshes** after alignment; the template is only a faint background. Lesion locations from the radiology report then sit on the real organ.
6. Quality check: mean landmark distance after alignment (e.g., < 15 mm); above the threshold → show only the "your CT" mode, never blended with the template.

27.4 Additional data fields

Field	Table	Meaning
`organ_status`	body_findings / organ_states	present, removed, absent_congenital, transplanted, implant_present
`variant_code`	organ_states	Anatomical variant code (if the record states one)
`device_type`	organ_states	Type of implanted device
`precision`	body_findings	Level of location detail
`template_id`	patient_profiles	Selected template (sex, age band) + scale parameters
`registration_error_mm`	ct_studies	CT registration error

27.5 Options by phase

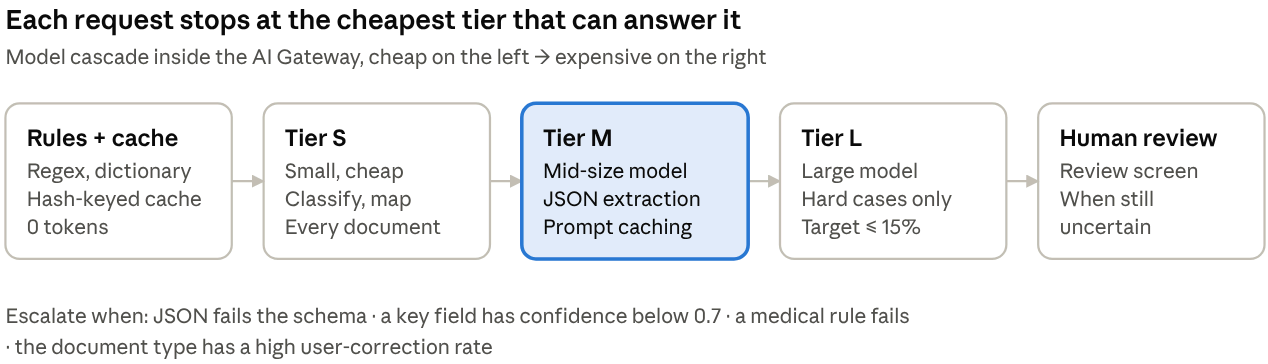
Option	Content	When
A — Fixed template + warnings	1 adult template + 2D; mismatches shown only as labels	MVP
B — Template library + organ status	Templates by sex/age band, scaled by height/weight, hidden removed organs, device icons, hierarchical regions	v1.5
C — Personal anatomy from CT	Meshes from the user's CT, landmark registration, compare mode with the template	v2

27.6 Testing

- "Special anatomy" test set: ≥ 30 synthetic records with removed organs, implants, variants, unknown side, organ-level-only locations.
- Viewer screenshots (visual regression) for every mismatch type.
- The Clinical Safety Officer approves how each type is displayed (hazard: the user misreads a location).

28. Model selection and token/cost optimisation

Strategy: **rules first, small models next, large models last**. OCR and segmentation run on self-hosted open models (0 tokens). LLMs only do work that needs language understanding, always through the AI Gateway so calls are cached, routed, limited and metered. Under the assumptions in 28.4, AI cost is about **0.01 USD per document**, roughly 10× below the 0.10 USD target.



model cascade · 5 tiers from cheap to expensive

The escalation rate to Tier L is the single most important cost metric; above 15%, improve the Tier M prompt/schema rather than accept the cost.

28.1 Model per task

Task	Default approach	Tier	Savings note
Image quality check	OpenCV	Rules	0 tokens
Printed OCR	PaddleOCR/VietOCR on CPU (GPU under load)	Self-hosted	0 tokens
Handwriting OCR	Vision LLM only on low-confidence crops	M	Send crops, not whole pages
Page classification	Keywords + small model; later a self-trained classifier	S	Send only the first 600 tokens of the page

Task	Default approach	Tier	Savings note
Field extraction	Structured-output LLM per Unit schema	M → L when needed	Cache system prompt + schema; only relevant pages; short JSON keys; output limits
ICD-10/LOINC normalisation	Dictionary → embedding kNN → small model picks from top-5	Rules → S	Self-hosted embeddings (bge-m3)
Region mapping	Rules → small model picks from the Unit's region list	Rules → S	Closed list, very short output
Text-description NER	Small model	S	Short input
RAG explanations	Mid-size model, answers ≤ 250 words	M	Cache the instructions; cache common questions per article
Intent guardrail	Rules + small model	Rules → S	Runs before any larger model
Offline judging, synthetic data	Large model	L	Nightly batch
CT segmentation	TotalSegmentator	Self-hosted GPU on demand	Scale to zero

28.2 Candidate models per tier (re-check catalogue and region before deciding)

Tier	Example candidates	Selection criteria
S	Small open models on Bedrock (e.g., Gemma, Ministral, small Qwen families) or self-hosted via vLLM/Ollama	Good-enough Vietnamese for classification/mapping on the dev set; lowest price

Tier	Example candidates	Selection criteria
M	Mid-size models with structured output and vision (e.g., the Claude Haiku line, mid-size Amazon Nova)	Extraction F1 on the golden set, prompt-caching support, available in the chosen region
L	Large models (e.g., the Claude Sonnet line)	Quality on hard cases; used only when Tier M fails
Embedding	Self-hosted bge-m3 / multilingual-e5, or a multilingual Bedrock embedding	Recall on Vietnamese questions

Choose by **bake-off**: run the same golden set on 2–3 candidates per tier, compare F1, latency and cost per document; write an ADR. Repeat quarterly or when new models appear.

28.3 Cost levers

Lever	Reference saving	Use in MediLens
Batch inference	About 50% vs on-demand for some models (Bedrock pricing)	Nightly evals, reprocessing after schema changes, synthetic data
Prompt caching	Cache reads about 90% cheaper than normal input in the pricing-page example; cache writes slightly dearer	System prompt + JSON Schema + fixed examples at the start of the prompt
Flex tier	50% below Standard for some providers	Non-urgent work
Cascade + escalation	Tier L share ⩽ 15%	AI Gateway
Result cache	100% for duplicate documents	Key = sha256(document) + prompt_version + model
Fewer input tokens	30–60% depending on the document (estimate)	Drop repeated headers/footers, send only clinical pages, cleaned OCR
Output limits	—	`max_tokens` per schema; explanations ⩽ 250 words

Lever	Reference saving	Use in MediLens
Quotas & budgets	Stops runaway cost	Per user per day, daily budget, circuit breaker
Distillation	Moves work from Tier M to S	Fine-tune a small model on user-confirmed outputs + synthetic data (once enough data passes G1-D)

28.4 Formula and cost example

```
C_doc = sum over steps s of r_s * (T_in_new * P_in + T_in_cached * P_cache + T_out * P_out) + G_sec * P_gpu
```

Here `r_s` is the share of documents passing step s, `T` token counts, `P` price per token, `G_sec` GPU seconds.

Example for a 3-page document (~2,500 OCR tokens). **Prices are illustrative assumptions:** Tier S 0.15/0.15 USD per million tokens (Ministral 8B in a US region on the Bedrock price page), Tier M 1/5 USD, Tier L 3/15 USD, cache read = 10% of the input price. Replace with real prices for your region.

Step	Tier	Share	Tokens (new in / cached in / out)	Cost (USD)
Page classification	S	100%	600 / 0 / 20	0.00009
Extraction	M	100%	2,500 / 1,500 / 800	0.00665
Mapping 5 findings	S	100%	1,000 / 0 / 100	0.00017
Escalation to Tier L	L	12%	4,000 / 0 / 800	0.00288
Total per document				**≈ 0.0098**
One explanation question	M	—	1,000 / 2,000 / 300	0.0027

With 100 users a month, each uploading 5 documents and asking 20 questions: about 4.9 + 5.4 ≈ **10 USD/month** in tokens. The MLOps skill's `cost_model.py` reproduces these numbers.

Is self-hosting worth it? An assumed 1 USD/hour GPU running all month costs about 720 USD. It only breaks even at about 720 / 0.0098 ≈ **73,000 documents/month** (before operations effort). At personal scale, pay-per-token APIs are far cheaper; self-host to

learn, or run GPU batches that scale to zero.

28.5 Three ways to run models

Option	Content	Pros	Cons	When
A — One API model	One mid-size model for everything	Simple	2–3× more expensive than B; single-vendor dependency	Foundation, prototype
B — Cascade through the AI Gateway (recommended)	Rules → S → M → L, cache, batch, quotas	Low cost, easy model swaps	Needs good evals to set thresholds	From Build MVP
C — Distil + self-host	Fine-tune a small extraction model, serve with vLLM/SageMaker or Bedrock Custom Model Import	Independent of token prices; data stays in the VPC	Fixed GPU cost, needs MLOps, needs enough licensed data	At large scale or when data must not leave the system

28.6 Maintaining models

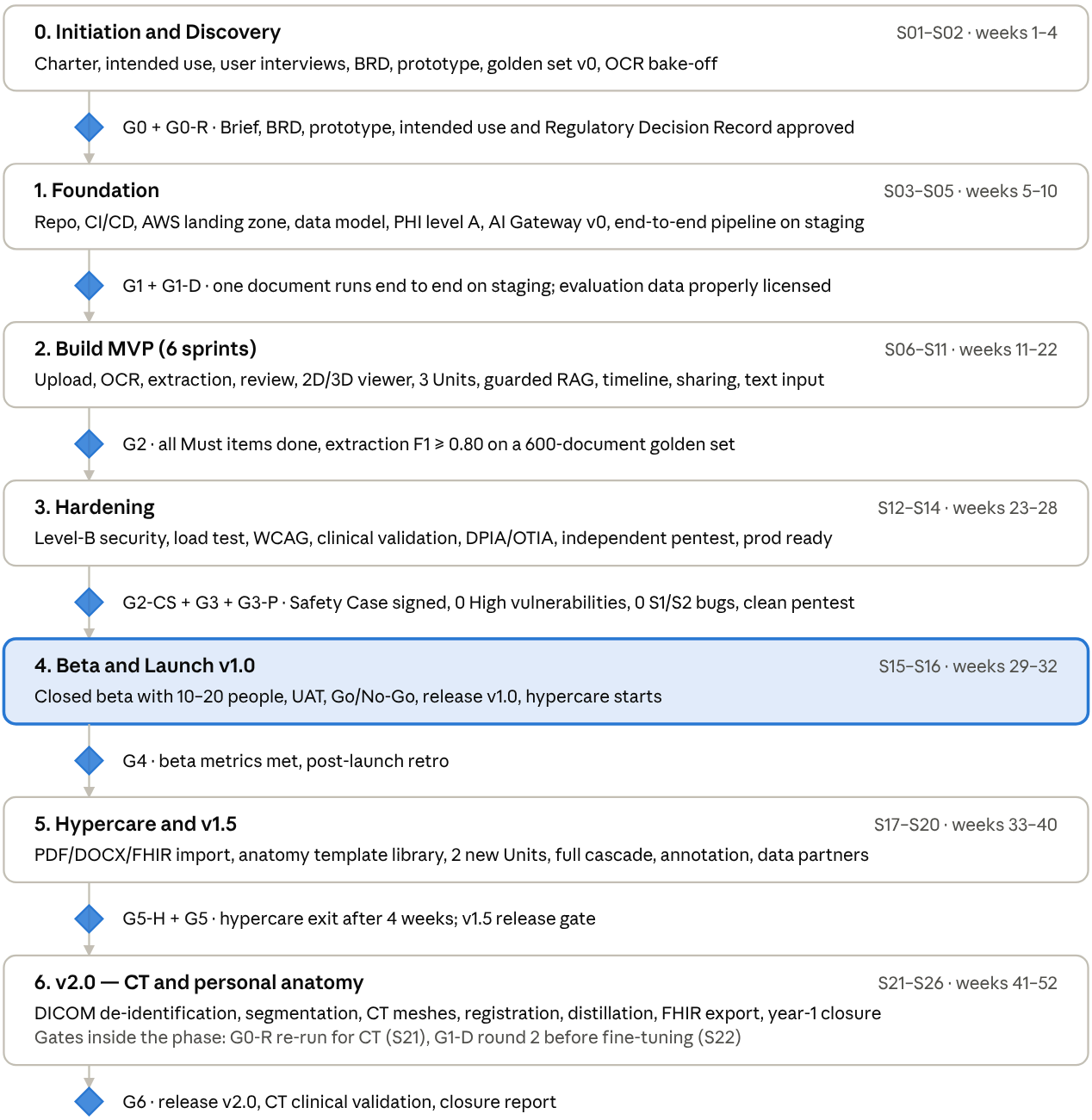
- **Model registry:** every configuration (provider, pinned model id, prompt version, escalation thresholds) has an id and a Model Card.
- **Quarterly bake-off**, and whenever a provider releases a new model or deprecates one.
- **Fallback provider** per tier, switchable via AppConfig in < 5 minutes.
- **Cost dashboard:** USD per document, escalation rate, cache hit rate, average tokens per step; alert when the daily budget is exceeded.
- **Thresholds:** cache hit ≥ 60% on fixed prompt parts; Tier L escalation ≤ 15%; cost ≤ 0.03 USD per document (long-term target in section 10).

29. Long-term sprint plan: 52 weeks, 26 sprints

This roadmap replaces the 32-week roadmap in section 5. The first five phases keep their length, so **v1.0 still launches in week 32**; after that come **Hypercare and v1.5** (weeks 33–40) and **v2.0 with CT images** (weeks 41–52). Each sprint is 10 working days × 4 hours (20 hours/week), 260 days and 1,040 hours in total. The work for every day (role, skill to call, deliverable, definition of done) is in the workbook **MediLens_Sprint_Plan_52w.xlsx** attached in the conversation.

v1.0 still launches in week 32; v2.0 with CT completes in week 52

26 sprints × 10 working days, 20 hours/week; diamond = gate to pass before the next phase



52-week roadmap · 7 phases, 14 gates

The five original gates G0–G4 stay. The five new gates from 22.4 sit in their sprints, plus G5 (release v1.5) and G6 (release v2.0). G0-R runs again in S21 before CT work; G1-D runs a second time in S22 before fine-tuning.

29.1 Sprint goals

Sprint	Weeks	Goal	Gate
S01	1–2	Initiate: charter, intended use, user research	
S02	3–4	Lock MVP scope, BRD, prototype, golden set v0, OCR choice	G0, G0-R
S03	5–6	Repo, CI, AWS landing zone, Terraform VPC, auth skeleton	
S04	7–8	Data model, level-A PHI boundary, secure upload, design system	
S05	9–10	End-to-end AI pipeline on staging, AI Gateway v0	G1, G1-D
S06	11–12	Upload, state machine, OCR with realtime status	
S07	13–14	LLM extraction, Review screen, model cascade v1	
S08	15–16	2D body viewer + Dental Unit	
S09	17–18	3D viewer + Cardiology Unit	
S10	19–20	Oncology Unit + guarded RAG explanations	
S11	21–22	Pre/post timeline, text descriptions, sharing, admin, data rights	G2
S12	23–24	Level-B security, performance, WCAG 2.2 AA	
S13	25–26	Clinical safety, regulatory, summative usability	G2-CS
S14	27–28	Independent pentest, UAT, production ready	G3, G3-P
S15	29–30	Closed beta with 10–20 people	
S16	31–32	Release v1.0	G4
S17	33–34	Hypercare + digital record import (PDF, DOCX, FHIR)	
S18	35–36	Anatomy template library + opt-in data donation	G5-H
S19	37–38	Respiratory and Nephrology–Urology Units + full model cascade	
S20	39–40	Annotation, data partnership, release v1.5	G5

Sprint	Weeks	Goal	Gate
S21	41–42	Regulatory re-assessment for CT + safe DICOM intake	G0-R (CT)
S22	43–44	Slice viewer, segmentation, meshes from CT	G1-D (round 2)
S23	45–46	Registration to the template + linking CT reports	
S24	47–48	Distillation + model cost optimisation	
S25	49–50	FHIR export, v2 hardening, CT clinical validation	
S26	51–52	Release v2.0 + year-1 closure	G6

29.2 Rhythm inside a sprint

- **Day 1:** 1-hour Sprint Planning (pick stories, write the Sprint Goal), then the day's main task.
- **Days 2–9:** one main task per day, wearing one role hat. The Skill column names the agent skill to call, e.g. `medilens-backend-engineer`; for multi-role work call `medilens-orchestrator`. A 5-minute daily note each morning.
- **Day 5:** an extra hour of refinement for the next sprint (BA hat).
- **Day 10:** Review (≤ 5-minute demo video), Retro, risk register update, gate check if the sprint has a gate; the rest is buffer for slipped work.
- **Recurring work** (weekly cost review, monthly steering report, quarterly licence review and model bake-off, DR drill every 6 months) is in the Recurring sheet and done alongside the main task.

29.3 What the workbook contains

Sheet	Use
Dashboard	Current sprint, today's task, next gate, % complete vs expected progress, overdue tasks
Settings	Start date and hours per day (yellow cells); every other date recalculates
Roadmap, Gantt	7 phases over 52 weeks, coloured by phase, ◆ marks weeks with a gate

Sheet	Use
Sprints	26 sprints: VI/EN goals, demo, gate, roles, key risk, hours and % done
Daily Plan	260 days: date, role, skill, VI/EN task, deliverable, definition of done; you update Status, Actual hours, Notes
Gates	14 gates: VI/EN criteria, evidence, approver, status
Roles	22 roles, their skills, planned days and hours
Recurring, Holidays, Change Log	Recurring work; 2026–2027 public holidays; baseline change log

Dates use the WORKDAY function and skip weekends and public holidays. Loaded holidays: 24/11/2026 (Vietnam Culture Day under Resolution 28/2026/QH16), New Year's Day, Lunar New Year 04–10/02/2027 (civil-service schedule announced by the Ministry of Home Affairs), Hung Kings' Commemoration 16/04/2027, 30/4, 1/5 and 02–03/09/2027. With the default start date of 19/10/2026 the plan ends on 01/11/2027. If your workplace follows a different holiday schedule, edit the Holidays sheet.

29.4 Time per role

In the baseline plan AI/ML takes the most hours (108, about 10%), followed by PM and Backend (100 each), Frontend (80) and QA (76). The nine new roles take about 27%: Clinical Safety 48 hours, Regulatory & Privacy 44, MLOps 40, Medical Imaging 32, Technical Writer 32, Data Curator 28, Support 24, Clinical Informaticist 20, 3D Medical Artist 16. The Roles sheet updates itself when you change a day's role.

29.5 When you fall behind

1. Use the day-10 buffer first.
2. More than 2 days late: cut the sprint's Should items (e.g., one fewer template variant); never cut tests, security or gates.
3. A whole sprint late: log a change request in the Change Log sheet before moving later sprints; never merge two gates into one.
4. Two sprints late in a row: the steering group reviews scope (risk R03 in section 11). The fallback is to keep v1.5 and move v2.0 to year 2.